

João Pedro Baza Garcia Rodrigues

Desenvolvimento de Software para Dimensionamento em Operações Unitárias

Campo Grande – MS

2026

João Pedro Baza Garcia Rodrigues

Desenvolvimento de Software para Dimensionamento em Operações Unitárias

Trabalho de Conclusão de Curso apresentado no Instituto de Química como requisito básico para a conclusão do Curso de Engenharia Química.

Universidade Federal de Mato Grosso do Sul – UFMS
Instituto de Química – INQUI
Engenharia Química

Orientador: Prof. Dr. Celso Murilo dos Santos

Campo Grande – MS
2026

João Pedro Baza Garcia Rodrigues

Desenvolvimento de Software para Dimensionamento em Operações Unitárias

Trabalho de Conclusão de Curso apresentado no Instituto de Química como requisito básico para a conclusão do Curso de Engenharia Química.

Trabalho aprovado. Campo Grande – MS, XX de novembro de 2026:

Prof. Dr. Celso Murilo dos Santos
Orientador

Prof. Dr. Nome do Professor
Convidado 1

Prof. Dr. Nome do Professor
Convidado 2

Campo Grande – MS
2026

Agradecimentos

Agradeço a Deus, por me conceder forças nos momentos difíceis e iluminar meu caminho ao longo desta jornada acadêmica.

Aos meus pais, Cassiano e Jacquecislene, por me darem não apenas a vida, mas também o dom do estudo e do senso crítico.

À minha namorada, Carolyne, que, com sua paciência, carinho e incentivo, foi essencial para que eu visse propósito no que eu fazia, como a luz que guia a direção da folha.

*“Não deseje que seja mais fácil, deseje ser melhor.
Não deseje menos problemas, deseje mais habilidades.
Não deseje menos desafios, deseje mais sabedoria.”*
(Jim Rohn)

Resumo

Este trabalho apresenta o desenvolvimento de um software de código aberto para dimensionamento e disponibilização de dados para materiais e compostos úteis na Engenharia Química. O software automatiza cálculos a partir de dados inseridos pelo usuário, utilizando equações consolidadas. O sistema permite integração via API (*Application Programming Interface*), promovendo flexibilidade, colaboração e apoio técnico no ensino e na prática da engenharia, contando com código aberto para colaboração e aprimoramento contínuo por parte da comunidade acadêmica.

Palavras-chave: Código aberto. Dimensionamento Industrial. Propriedade de compostos. Características de acessórios em tubulações.

Lista de ilustrações

Figura 1 – Interface gráfica do módulo de cálculos de tubulação (<i>Piping</i>)	24
Figura 2 – Código da função <i>get_calculated_diameter()</i>	27
Figura 3 – Código da função <i>get_real_diameter()</i>	28
Figura 4 – Código da função <i>determine_limiting_reagent()</i> – Parte 1/2	30
Figura 5 – Código da função <i>determine_limiting_reagent()</i> – Parte 2/2	31
Figura 6 – Código da função <i>_calculate_concentration_and_rate()</i> – Trecho da velocidade da reação	32
Figura 7 – Código da função <i>_calculate_concentration_and_rate()</i> – Trecho da concentração dos componentes	33
Figura 8 – Código da função <i>_calculate_dilution_factor()</i>	34
Figura 9 – Código da função <i>_conversion_and_kinetics_in_cstr()</i> – Parte 1/2	36
Figura 10 – Código da função <i>_conversion_and_kinetics_in_cstr()</i> – Parte 2/2	37
Figura 11 – Código da função <i>_initialize_reactor()</i> – Parte 1/2	38
Figura 12 – Código da função <i>_initialize_reactor()</i> – Parte 2/2	39
Figura 13 – Código da função <i>_volume_and_kinetics_in_cstr()</i> – Parte 1/2	40
Figura 14 – Código da função <i>_volume_and_kinetics_in_cstr()</i> – Parte 2/2	41
Figura 15 – Código da função <i>_residence_time_and_kinetics_in_cstr()</i> – Parte 1/2	42
Figura 16 – Código da função <i>_residence_time_and_kinetics_in_cstr()</i> – Parte 2/2	43
Figura 17 – Código da função objetivo para o PFR	44
Figura 18 – Especificações <i>hardcoded</i> para tubulações	46
Figura 19 – Cálculo da perda de carga por Darcy-Weisbach com acessórios	47
Figura 20 – Resultados do balanço de massa com reciclo e reação	49
Figura 21 – Cálculo do Reynolds – Parte 1/3	51
Figura 22 – Cálculo do Reynolds – Parte 2/3	52
Figura 23 – Cálculo do Reynolds – Parte 3/3	53
Figura 24 – Interface de cálculo do fator de atrito	54
Figura 25 – Cálculo do fator de atrito – Parte 1/2	55
Figura 26 – Cálculo do fator de atrito – Parte 2/2	56
Figura 27 – Cálculo da perda de carga – Parte 1/3	58
Figura 28 – Cálculo da perda de carga – Parte 2/3	59
Figura 29 – Cálculo da perda de carga – Parte 3/3	60
Figura 30 – Cálculo do <i>head</i> – Parte 1/2	61
Figura 31 – Cálculo do <i>head</i> – Parte 2/2	62
Figura 32 – Cálculo do <i>NPSH</i> – Parte 1/2	63
Figura 33 – Cálculo do <i>NPSH</i> – Parte 2/2	64
Figura 34 – Tipos de dutos de escoamento	65

Figura 35 – Exemplo da identificação e critérios de diâmetro hidráulico	66
Figura 36 – Obtenção de propriedades pela biblioteca <i>CoolProp</i>	68
Figura 37 – Obtenção de propriedades críticas pela biblioteca <i>CoolProp</i>	69
Figura 38 – Interface de cálculo para as propriedades de uma mistura	70
Figura 39 – Obtenção de propriedades de misturas pela biblioteca <i>CoolProp</i> – Parte 1/2	71
Figura 40 – Obtenção de propriedades de misturas pela biblioteca <i>CoolProp</i> – Parte 2/2	72
Figura 41 – Inicialização da biblioteca de componentes via <i>Python</i>	74
Figura 42 – Inicialização da biblioteca de componentes via <i>frontend</i>	74
Figura 43 – Listagem parcial de componentes disponíveis na biblioteca termodinâmica	75
Figura 44 – Propriedades termofísicas da água calculadas a 300K e 101,3kPa. . . .	75
Figura 45 – Mapeamento de chaves de propriedades e suas unidades padronizadas.	76
Figura 46 – Propriedades críticas e do ponto triplo calculadas para a água.	77
Figura 47 – Propriedades termodinâmicas disponíveis para cálculo em misturas. . .	78
Figura 48 – Propriedades termodinâmicas disponíveis para cálculo em misturas. . .	79
Figura 49 – Código de validação do Balanço de Massa (Misturador)	85
Figura 50 – Resultados expandidos da validação de Balanço de Massa (Misturador, Reator e Splitter)	88
Figura 51 – Lista parcial de acessórios disponíveis (JSON)	89
Figura 52 – Especificações do acessório Cotovelo 90º Raio Longo (JSON)	90
Figura 53 – Lista parcial de materiais de tubulação (JSON)	91
Figura 54 – Especificações do material Aço Comercial (JSON)	92
Figura 55 – Lista de classes de pressão (Schedules) disponíveis (JSON)	92
Figura 56 – Lista parcial de diâmetros para SCH40 (JSON)	94
Figura 57 – Especificações do tubo SCH40 DN50 (JSON)	95

Lista de tabelas

Tabela 1	– Bibliotecas e <i>frameworks</i> utilizados no <i>backend</i>	21
Tabela 2	– Bibliotecas e <i>frameworks</i> utilizados no <i>frontend</i>	24
Tabela 3	– Validação Balanço de Massa (Misturador)	86
Tabela 4	– Validação CSTR ($X = 0,85$)	96
Tabela 5	– Validação PFR ($X = 0,90$)	97
Tabela 6	– Validação PFR com Reciclo ($R = 2,0; X = 0,90$)	98

Lista de abreviaturas e siglas

API	Application Programming Interface
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
EDO	Equação diferencial ordinária
UI	User Interface
UX	User Experience
PFR	Plug Flow Reactor
CSTR	Continuous Stirred Tank Reactor
T	Temperatura
P	Pressão
NPSH	Net Positive Suction Head

Sumário

1	INTRODUÇÃO	17
2	JUSTIFICATIVA	18
3	OBJETIVOS	19
3.1	Geral	19
3.2	Específico	19
4	METODOLOGIA	21
4.1	Desenvolvimento do Software	21
4.1.1	<i>Backend</i>	21
4.1.2	<i>Frontend</i>	23
4.1.3	<i>Deploy</i>	24
4.1.4	Versionamento	25
4.2	Funções de Engenharia	25
4.2.1	Dimensionamento de Tubulações	25
4.2.1.1	Diâmetro Calculado	26
4.2.1.2	Diâmetro Real	27
4.2.2	Reatores	28
4.2.2.1	Reator do Tipo CSTR	34
4.2.2.1.1	Conversão e Cinética (Cálculo do Volume)	35
4.2.2.1.2	Volume e Cinética (Cálculo da Conversão)	39
4.2.2.1.3	Tempo de Residência e Cinética	41
4.2.2.2	Reator do Tipo PFR	43
4.2.3	Tubulações	44
4.2.4	Balanco de Massa	48
4.2.5	Escoamento, Perdas de Carga e Diâmetros Hidráulicos	49
4.2.5.1	Número de Reynolds	50
4.2.5.2	Fator de Atrito	53
4.2.5.3	Perda de Carga – Darcy-Weisbach e Hazen-Williams	56
4.2.5.4	Altura Manométrica entre Dois Pontos	60
4.2.5.5	NPSH Disponível	62
4.2.5.6	Diâmetro Hidráulico para Geometrias Diversas	64
4.2.6	Propriedades Termofísicas	66
4.3	API	73
4.4	Validação de Cálculos e Especificação de Retornos	73

4.4.1	Componentes	73
4.4.1.1	Listar Componentes	74
4.4.1.2	Valor de Propriedade	75
4.4.1.3	Nome de Propriedades	76
4.4.1.4	Propriedades Críticas	76
4.4.1.5	Opções de Propriedades de Misturas	77
4.4.1.6	Valor de Propriedades de Misturas	78
4.4.2	Hidráulicos	79
4.4.2.1	Perda de Carga	79
4.4.2.2	Reynolds	80
4.4.2.3	Fator de Atrito	80
4.4.2.4	Diâmetro Real	81
4.4.2.5	Diâmetro Calculado	81
4.4.2.6	NPSH Disponível	82
4.4.2.7	Head	82
4.4.2.8	Diâmetro Hidráulico	82
4.4.3	Balanços de Massa	83
4.4.3.1	Misturador Ideal	83
4.4.3.2	Reator de Conversão	86
4.4.3.3	Divisor de Corrente (Splitter)	86
4.4.4	Tubulações	89
4.4.4.1	Acessórios	89
4.4.4.2	Especificação de Acessórios	90
4.4.4.3	Composição	90
4.4.4.4	Especificação da Composição	91
4.4.4.5	Schedules	92
4.4.4.6	Diâmetros	93
4.4.4.7	Especificação de Diâmetros	95
4.4.5	Reatores	95
4.4.5.1	CSTR: Reator Tanque Agitado Continuamente	96
4.4.5.2	PFR: Reator Tubular	96
4.4.5.3	PFR com Reciclo	97
5	RESULTADOS E DISCUSSÕES	99
6	CONCLUSÕES	101
	REFERÊNCIAS	103

1 Introdução

O dimensionamento de equipamentos em processos químicos é uma etapa crítica no projeto de plantas industriais, envolvendo princípios fundamentais da engenharia química, como balanços de massa e energia, termodinâmica, transferência de calor e de massa e cinética química. Esses conceitos são aplicados para definir características operacionais e tamanhos de equipamentos, como trocadores de calor, colunas de destilação, reatores, bombas e compressores. O domínio das operações unitárias é essencial para garantir desempenho seguro, eficiente e otimizado em termos de custos nos processos industriais (FELDER; ROUSSEAU, 2005).

Apesar de *softwares* consagrados como *Aspen Plus*®, *HYSYS*® e *ChemCAD*® serem amplamente utilizados na indústria, seu uso é limitado por custos elevados de licenciamento e pela falta de acesso em ambientes acadêmicos (Aspen Technology, 2025; Chemstations, 2025; ALVES et al., 2022).

Essas ferramentas, embora poderosas, nem sempre priorizam o aprendizado conceitual, o que pode dificultar a compreensão dos cálculos pelos estudantes e engenheiros em formação. Há, portanto, espaço para soluções acessíveis, flexíveis e didáticas. Este trabalho propõe o desenvolvimento de um *software web* próprio e gratuito para simulação e dimensionamento de operações unitárias, integrando conhecimento teórico com programação moderna.

2 Justificativa

O desenvolvimento de um *software* gratuito para dimensionamento de operações unitárias integra a teoria da engenharia química com ferramentas de programação e automação, democratizando o acesso a simulações para instituições de ensino, profissionais autônomos e pequenas empresas desprovidas de licenças comerciais. Isso favorece o aprendizado ativo, permitindo visualizações em tempo real de como parâmetros operacionais afetam o desempenho dos equipamentos.

No contexto da Indústria 4.0, o domínio de linguagens de programação e bancos de dados é um diferencial para engenheiros químicos, dada a crescente digitalização de processos e a necessidade de análise de dados em tempo real.

Softwares proprietários como *Aspen Plus*® e *HYSYS*® apresentam custo elevado e acesso restrito ([Aspen Technology, 2025](#); [ALVES et al., 2022](#)), enquanto alternativas *open-source* como o nacional *DWSIM* ([DWSIM, 2025](#)) ainda apresentam limitações no acoplamento de módulos de forma didática e customizável. Planilhas em *Excel*® ou rotinas em *MATLAB*®, por sua vez, oferecem baixa modularização e carecem de interfaces amigáveis para simulação de processos complexos. Identifica-se, assim, uma lacuna em soluções abertas, escaláveis e focadas no aprendizado, que combinem rigor técnico com facilidade de uso. Este trabalho busca preencher essa lacuna com um sistema modular e extensível, servindo tanto ao ensino quanto à prática profissional.

Sendo de código aberto, espera-se que estudantes com formação interdisciplinar em engenharia química e desenvolvimento de *software* possam contribuir para expandir e aprimorar o repertório da ferramenta, servindo como precursora para uso em maior escala.

3 Objetivos

3.1 Geral

Desenvolver um *software web* completo e modular para cálculo e dimensionamento de equipamentos de operações unitárias, integrando rotinas de balanço de massa, transferência de calor e reações químicas. O sistema deve adotar arquitetura cliente-servidor com interface *web* acessível por meio de API aberta, mantendo o código disponível publicamente.

3.2 Específico

- Implementar rotinas de cálculo baseadas em equações clássicas para operações unitárias como reatores, tubulações e demais correlatos, com perda de carga, *NPSH* etc;
- Desenvolver uma interface *web* interativa e responsiva para inserção e interpretação de dados em diversos dispositivos, favorecendo a experiência do usuário;
- Integrar um banco de dados local (inicialmente *hardcoded*) com propriedades físico-químicas de substâncias comuns (coeficientes de difusão, calor específico, entalpia de vaporização etc.);
- Validar os resultados comparando-os com dados da literatura, assegurando precisão e confiabilidade dos cálculos.

4 Metodologia

4.1 Desenvolvimento do Software

A metodologia adotada combina fundamentos da engenharia química com técnicas modernas de desenvolvimento de *software*, visando garantir precisão nos cálculos e acessibilidade por meio de uma interface *web* intuitiva. O projeto foi estruturado em etapas que abrangem desde o levantamento bibliográfico das equações de dimensionamento até a implementação e validação do sistema completo. O desenvolvimento foi dividido em quatro partes principais: *backend*, *frontend*, *deploy* e versionamento.

4.1.1 *Backend*

O *backend* foi desenvolvido em *Python*, utilizando o *framework FastAPI*, escolhido por sua alta *performance*, suporte a tipagem forte e facilidade na criação de APIs RESTful. Essa camada é responsável por processar os cálculos, gerenciar as rotas de comunicação (*endpoints*) e servir os resultados ao *frontend*. Além disso, por se tratar de uma linguagem de alto nível, sua sintaxe é simples e intuitiva para iniciantes.

A Tabela 1 resume as principais bibliotecas e *frameworks* utilizados, sua funcionalidade e a correspondência com aplicações típicas da engenharia química.

Tabela 1 – Bibliotecas e *frameworks* utilizados no *backend*

Nome	Funcionalidade	Como se enquadra no uso da Engenharia Química
FastAPI	Framework web moderno para criação de <i>APIs RESTful</i> em <i>Python</i> , com validação automática de dados e documentação interativa.	<i>Backend</i> para expor os cálculos de engenharia química através de <i>endpoints</i> HTTP, permitindo que a interface web ou outros sistemas acessem as funções de dimensionamento, balanço de massa, cálculos de reatores etc.
<i>Uvicorn</i>	Servidor ASGI para rodar aplicações <i>web</i> assíncronas em <i>Python</i> .	Executa o servidor da <i>API FastAPI</i> , processando as requisições dos cálculos.

(Continua na próxima página)

Tabela 1 – Bibliotecas e *frameworks* utilizados no *backend*
(continuação)

Nome	Funcionalidade	Como se enquadra no uso da Engenharia Química
<i>Pydantic</i>	Biblioteca para validação de dados e gerenciamento de configurações usando <i>Python type hints</i> .	Valida os dados de entrada dos cálculos (vazões, temperaturas, pressões, propriedades físicas etc.), garantindo que os parâmetros fornecidos estejam no formato e tipo corretos antes de executar os cálculos.
<i>CoolProp</i>	Biblioteca especializada em propriedades termodinâmicas de fluidos, com banco de dados extenso de substâncias.	Cálculo de propriedades termodinâmicas (densidade, viscosidade, entalpia, entropia, calor específico etc.) de compostos e misturas.
<i>Pint</i>	Gerenciamento de unidades físicas, conversões e cálculos dimensionais	Permite trabalhar com diferentes sistemas de unidades, garantindo consistência dimensional nos cálculos.
<i>Matplotlib</i>	Biblioteca de visualização de dados e criação de gráficos 2D/3D.	Gera gráficos de processos químicos como curvas de reação, perfis de temperatura, diagramas de fases, curvas de bomba, e visualizações de resultados de simulações.
<i>NumPy</i>	Biblioteca fundamental para computação científica com <i>arrays</i> multidimensionais e operações matemáticas vetorizadas	Realiza cálculos matriciais, interpolações, operações vetoriais para balanços de massa e energia, resoluções de sistemas de equações lineares em processos químicos
<i>SciPy</i>	Conjunto de algoritmos científicos para otimização, integração numérica, álgebra linear, estatística	Resolução de EDOs, otimiza processos químicos, realiza integrações numéricas para cálculos de conversão, ajuste de curvas experimentais

(Continua na próxima página)

Tabela 1 – Bibliotecas e *frameworks* utilizados no *backend*
(continuação)

Nome	Funcionalidade	Como se enquadra no uso da Engenharia Química
<i>Pytest</i>	<i>Framework</i> de testes, com suporte a testes unitários, testes de integração, <i>fixtures</i> , parametrização e relatórios detalhados.	Permite validar automaticamente a confiabilidade dos cálculos de engenharia química, garantindo que equações, modelos matemáticos e rotinas numéricas estejam corretos e continuem funcionando após alterações no código.

Fonte: Autoria própria.

4.1.2 Frontend

O *frontend* corresponde à interface gráfica do usuário, sendo a parte visual e interativa do sistema, acessada diretamente pelo navegador. Ele é responsável por exibir os resultados dos cálculos, receber dados de entrada e enviar as informações ao servidor (*backend*).

Foi utilizado HTML, CSS e JS puros, com bibliotecas, *frameworks* e *plugins* modernos para melhorar UI e UX. O JS foi criado de forma modular, de forma que cada aba da interface contém classes próprias, melhorando legibilidade e organização, além de um específico para UI, padronizando retornos, *inputs* e telas. A Figura 1 ilustra a interface de usuário desenvolvida para o módulo de Piping, evidenciando as escolhas de *design* descritas.

Figura 1 – Interface gráfica do módulo de cálculos de tubulação (*Piping*)

Chemical Engineering Calculator
Comprehensive toolkit for chemical engineering calculations

Piping | Sizing | Flow | Pump | Components | Reactor | Mass Balance

Piping Calculations

Composition Selection

Select Composition
Commercial steel

Composition Details

Property	Value	Units
Name	Commercial steel	-
Description	Standard carbon steel piping used in many industrial applications.	-
Applications	Water, gas, oil, steam transport in industrial settings.	-
Specifications - Roughness	0.0600000	millimeter
Specifications - Roughness Coefficient	135.000	dimensionless

Schedule & Diameter

Select Schedule
SCH10

Select Diameter
6 mm

Diameter Details

Property	Value	Units
External Diameter	10.3000	millimeter
Thickness	1.24500	millimeter
Weight	0.277000	kilogram / meter

Fonte: Autoria própria.

Tabela 2 – Bibliotecas e *frameworks* utilizados no *frontend*

Nome	Funcionalidade
<i>Tailwind</i>	<i>Framework</i> utilitário moderno que facilita a criação de interfaces responsivas e personalizadas com classes pré-definidas de estilo.
<i>Select2</i>	<i>Plugin JavaScript</i> que melhora campos do tipo <i>select</i> com busca, múltipla seleção e interface amigável.
<i>SweetAlert2</i>	Biblioteca para exibir alertas e mensagens personalizadas com <i>design</i> moderno e interativo na forma de <i>popup</i> .

Fonte: Autoria própria.

4.1.3 Deploy

O *deploy* do sistema foi realizado por meio de *Docker*, tecnologia que permite empacotar a aplicação e suas dependências em contêineres portáteis e reproduzíveis. Essa abordagem elimina problemas de compatibilidade entre ambientes e facilita o uso do *software* em diferentes sistemas operacionais.

Foram criadas imagens separadas para o *backend* e o *frontend* (estático), que podem ser executadas em conjunto ou individualmente. O uso do *Docker* também simplifica futuras implementações em servidores remotos e integração com orquestradores, garantindo escalabilidade e isolamento dos serviços. A aplicação encontra-se disponível para acesso público em <https://tcc.homelab.sistemasj.com.br>.

4.1.4 Versionamento

O controle de versão foi implementado com o *Git* para rastrear alterações no código-fonte, permitir colaboração e garantir segurança no desenvolvimento. O repositório foi criado no *GitHub* como público, permitindo a colaboração entre diferentes contribuidores. O uso de versionamento também favorece a rastreabilidade científica do projeto, assegurando transparência e reprodutibilidade dos resultados obtidos. O repositório original pode ser acessado em <https://github.com/joao-baza/TCC>.

4.2 Funções de Engenharia

Para cada função existente no projeto, seus *inputs*, *outputs* e definições do que se retornam estão definidos no que se segue, sendo alguns responsáveis por informações contidas no banco de dados local ou presente nas bibliotecas em utilização, enquanto outros são responsáveis por calcular parâmetros de projeto.

4.2.1 Dimensionamento de Tubulações

Tubulações são componentes por onde o fluido escoar, permitindo a transferência de matéria entre equipamentos, desde a entrada de matéria-prima até a saída de produtos. Seu correto dimensionamento é imprescindível para a segurança e a eficiência do processo.

O dimensionamento consiste em definir o diâmetro nominal e o *schedule* capazes de conduzir as vazões dentro de faixas de velocidade e perda de carga economicamente ótimas. As grandezas envolvidas incluem vazão (mássica ou volumétrica), velocidade admissível, propriedades do fluido (massa específica, viscosidade), comprimento equivalente (devido a acessórios) e características do material (rugosidade, pressão de projeto).

Na prática, calcula-se um diâmetro a partir da vazão e velocidade alvo e seleciona-se o diâmetro comercial imediatamente superior disponível, verificando em seguida a perda de carga pelo método de Darcy–Weisbach e a integridade mecânica conforme normas aplicáveis. Temperatura e pressão afetam propriedades físicas do fluido e critérios de espessura da tubulação; adicionalmente, a topografia da planta e o número de acessórios elevam o comprimento equivalente total.

Subdimensionar uma linha implica grandes perdas de carga e custos energéticos; superdimensionar acarreta CAPEX elevado e pode gerar velocidades muito baixas (favorecendo sedimentação em linhas com sólidos). O embasamento teórico combina a equação de continuidade, correlações de atrito e seleção de materiais, apoiado no uso do diagrama de Moody (WHITE, 2018).

4.2.1.1 Diâmetro Calculado

O diâmetro calculado é baseado na vazão e na velocidade média do fluido escoando em um duto circular, de acordo com a Equação 4.1:

$$D = \sqrt{\frac{4 \cdot Q}{\pi \cdot V}} \quad (4.1)$$

Dos quais,

- Q – Vazão volumétrica do fluido;
- V – Velocidade do fluido;
- D - Diâmetro hidráulico do duto circular.

Essa relação advém diretamente da equação da continuidade para escoamento em regime permanente (WHITE, 2018). No código, a função *get_calculated_diameter* presente na classe dos cálculos hidráulicos utiliza o controle de unidades via *Pint* e retorna o resultado em milímetros, assegurando consistência dimensional.

Figura 2 – Código da função `get_calculated_diameter()`

```

def get_calculated_diameter(self, parameters: Dict[str, object]):
    """
    Hydraulic diameter from flow rate and velocity.

    Parameters
    -----
    parameters : dict
        ``{"flow_rate": <m³/s>, "velocity": <m/s>}``

    Returns
    -----
    pint.Quantity
        Calculated diameter (mm).
    """
    req = ["flow_rate", "velocity"]
    self._require_keys(parameters, req)
    self._validate_numeric(parameters, req)

    Q = parameters["flow_rate"] * self.ureg.m**3 / self.ureg.s
    V = parameters["velocity"] * self.ureg.m / self.ureg.s

    # Use **0.5 instead of math.sqrt to keep units intact
    D = (4 * Q / (np.pi * V)) ** 0.5
    D_mm = D.to(self.ureg.mm)

    if D_mm <= 0 * self.ureg.mm:
        raise ValueError(f"Diameter must be positive, got {D_mm}.")

    return D_mm

```

Fonte: Autoria própria.

4.2.1.2 Diâmetro Real

O diâmetro real corresponde ao diâmetro nominal da tubulação a ser utilizada. Fabricantes produzem tubos em geometrias padronizadas (padrões de espessura como SCH 10, SCH 40, SCH 80 etc.). Deve-se calcular inicialmente o diâmetro necessário pelo método anterior e então escolher, em tabela de catálogo, a tubulação comercial com diâmetro interno imediatamente superior, no material e *schedule* desejados. Essa lógica é implementada na função `get_real_diameter`, garantindo que o diâmetro selecionado comportará a vazão com velocidade dentro do admissível. Após a seleção, verifica-se a perda de carga resultante e, se necessário, itera-se para outro diâmetro.

Figura 3 – Código da função `get_real_diameter()`

```
def get_real_diameter(self, parameters: Dict[str, object]):  
    """  
    Pick the next available nominal diameter above the calculated one.  
  
    Parameters  
    -----  
    parameters : dict  
        ``{"calculated_diameter": <mm>, "schedule": <str>}``  
  
    Returns  
    -----  
    pint.Quantity  
        Nominal diameter (mm).  
    """  
    req = ["calculated_diameter", "schedule"]  
    self._require_keys(parameters, req)  
    self._validate_numeric(parameters, ["calculated_diameter"])  
  
    d_calc = parameters["calculated_diameter"]  
    schedule = parameters["schedule"]  
  
    diameters = sorted(Piping().diameters(schedule))  
    for dn in diameters:  
        if dn > d_calc:  
            return dn * self.ureg.mm  
  
    raise ValueError(  
        f"No diameter in schedule '{schedule}' exceeds {d_calc} mm."  
    )
```

Fonte: Autoria própria.

4.2.2 Reatores

Reatores são equipamentos projetados para promover reações químicas, convertendo reagentes em produtos sob condições controladas de operação.

Modelos ideais como CSTR, PFR e batelada fornecem limites úteis para projeto e análise de desempenho. As variáveis centrais envolvidas no projeto de reatores incluem conversão dos reagentes, seletividade e rendimento (em caso de múltiplas reações), velocidade de reação, tempo de residência, volume do reator e condições de operação (temperatura, pressão). Industrialmente, o correto dimensionamento de reatores afeta diretamente a capacidade produtiva, a qualidade do produto e a segurança, especialmente no controle de remoção de calor em reações exotérmicas. A cinética química depende fortemente da temperatura (lei de Arrhenius) e da forma funcional da taxa de reação (ordem de reação,

efeitos de inibição, presença de inertes, variação volumétrica em fase gasosa etc.). Erros na previsão de conversão ou no cálculo do volume reacional podem levar a baixa seletividade, formação de pontos quentes (hot spots) e riscos operacionais. A base teórica conjuga balanços de massa por componente com leis cinéticas apropriadas, integradas no espaço (PFR) ou no tempo de residência (CSTR), podendo incluir balanço de energia quando o regime térmico é relevante (FOGLER, 2009; LEVENSPIEL, 2000).

No escopo deste projeto, são considerados reatores isotérmicos e monofásicos (todos os reagentes e produtos em uma única fase, líquida ou gasosa). O código rejeita misturas de componentes em estados físicos diferentes, a fim de garantir a validade dos balanços e do fator de expansão volumétrica utilizado para o estado atual dos módulos de cálculo para reatores.

Algumas lógicas gerais se aplicam a todos os tipos de reatores implementados. Por exemplo, a determinação do reagente limitante, onde: identifica-se aquele reagente cuja razão entre a vazão molar de entrada e seu coeficiente estequiométrico é a menor dentre os reagentes presentes. Em termos de cálculo, percorrem-se os reagentes (coeficientes estequiométricos negativos), calculando conforme a Equação 4.2:

$$razao_i = \frac{N_{i0}}{N_{u0}} \quad (4.2)$$

, onde N_{i0} é a entrada molar do componente i e N_{u0} seu coeficiente estequiométrico. O reagente com menor valor de $razao_i$ é o limitante (FELDER; ROUSSEAU, 2005).

Figura 4 – Código da função *determine_limiting_reagent()* – Parte 1/2

```
def determine_limiting_reagent(self, parameters):  
    """Determine the limiting reagent based on stoichiometric coefficients and  
    component concentrations."""  
    self._require_keys(parameters, ["components", "stoichiometric_coefficients"])  
  
    components = parameters["components"]  
    stoichiometric_coefficients = parameters["stoichiometric_coefficients"]  
  
    if len(components) != len(stoichiometric_coefficients):  
        raise ValueError("Components and stoichiometric coefficients must have  
the same length.")  
  
    required_keys = ["flow_rate_inlet", "molar_concentration_inlet",  
"component_name"]  
    for component in components:  
        for key in required_keys:  
            if key not in component:  
                raise KeyError(f"Missing key '{key}' in component {component}")  
  
    limiting_index = None  
    min_ratio = float('inf')  
  
    ...
```

Fonte: Autoria própria.

Figura 5 – Código da função *determine_limiting_reagent()* – Parte 2/2

```

def determine_limiting_reagent(self, parameters):
    """Determine the limiting reagent based on stoichiometric coefficients and
    component concentrations."""
    ...

    for i, component in enumerate(components):
        coef = stoichiometric_coefficients[i]

        if coef == 0:
            raise ValueError(f"Stoichiometric coefficient for component
{component['component_name']} is zero.")

        if coef < 0: # It's a reactant
            flow_rate = component["flow_rate_inlet"] * self.ureg.m**3 /
self.ureg.s
            molar_concentration = (component["molar_concentration_inlet"] *
self.ureg.mol / self.ureg.L).to(self.ureg.mol / self.ureg.m**3)

            ratio = (flow_rate * molar_concentration / abs(coef)).magnitude #
mol/s per mol

            if ratio < min_ratio:
                min_ratio = ratio
                limiting_index = i # Store the index of the limiting reagent

    if limiting_index is None:
        raise ValueError("No limiting reagent found. Check stoichiometric
coefficients and components.")

    return limiting_index

```

Fonte: Autoria própria.

A velocidade de reação é modelada, em geral, por uma lei de potência do tipo:

$$r = k \cdot \prod C_i^{m_i} \quad (4.3)$$

Dos quais,

- $C_i^{m_i}$ – Concentração de um reagente i elevado à sua ordem de velocidade em uma dada reação
- r – taxa de reação por volume;
- k constante de velocidade

Figura 6 – Código da função `_calculate_concentration_and_rate()` – Trecho da velocidade da reação

```
def _calculate_concentration_and_rate(self, components,
    stoichiometric_coefficients, reaction_rate_params,
    limiting, C0_i, C_lim0, X, dilution_factor):
    """Calculate the reactor concentrations and reaction rate for a given
    conversion."""
    # --[Cálculo de concentrações omitidas para reduzir imagem]--

    # Calculate the reaction rate  $r = k \prod C_i^{n_i}$ 
    rate_expression = 1.0 # Initialize variable
    for i, n_i in enumerate(reaction_rate_params["reaction_orders"]):
        if n_i != 0: # Only consider terms with non-zero order
            rate_expression *= concentration_values[i] ** n_i

    k_unit = self.ureg.Unit('mol/m³/s') / rate_expression.units
    k = reaction_rate_params["k"] * k_unit
    r = k * rate_expression

    return outlet_concentrations, k, r
```

Fonte: Autoria própria.

A integração da cinética com a estequiometria é feita via conversão X do reagente limitante. Os balanços estequiométricos relacionam as concentrações de saída de cada componente à conversão X . Para cada componente i na reação, têm-se a seguinte relação geral, sendo o valor do mais ou menos positivo para formação (produtos), negativo para consumo (reagentes) e, claro, uma conversão igual a zero para inertes.

$$C_i = \frac{C_{i0} \pm \left(\frac{n_i}{n_A} \cdot C_A \cdot X\right)}{\varepsilon} \quad (4.4)$$

Dos quais,

- C_i – Concentração do componente na saída
- C_{i0} – Concentração do componente na entrada
- ε – Coeficiente de expansão volumétrico
- X – Conversão do reagente limitante
- C_A – Concentração do reagente limitante
- n_i – Coeficiente estequiométrico do componente
- n_A – Coeficiente estequiométrico do reagente limitante

Figura 7 – Código da função `_calculate_concentration_and_rate()` – Trecho da concentração dos componentes

```
def _calculate_concentration_and_rate(self, components,
    stoichiometric_coefficients, reaction_rate_params,
    limiting, C0_i, C_lim0, X, dilution_factor):
    """Calculate the reactor concentrations and reaction rate for a given
    conversion."""
    # --[Cálculo da velocidade da reação omitida para reduzir imagem]--

    outlet_concentrations = {}
    concentration_values = []
    for i in range(len(components)):
        if stoichiometric_coefficients[i] < 0: # reactant
            if i == limiting:
                # For the limiting reactant
                Ci = C0_i[i] * (1 - X) / dilution_factor
            else:
                # For other reactants
                theta_i = C0_i[i]/C_lim0
                stoichiometric_ratio = abs(stoichiometric_coefficients[i]) /
                abs(stoichiometric_coefficients[limiting])
                X_i = X / stoichiometric_ratio
                if X_i > 1: # Cannot have more than 100% conversion
                    X_i = 1
                Ci = C0_i[i] * (1 - X_i) / dilution_factor
        else:
            if stoichiometric_coefficients[i] > 0: # product
                # For products
                stoichiometric_ratio = stoichiometric_coefficients[i] /
                abs(stoichiometric_coefficients[limiting])
                # Ensure C_lim0 is a single value, not a sequence
                lim_conc = C_lim0 if isinstance(C_lim0, (int, float)) or
                hasattr(C_lim0, 'magnitude') else C_lim0[0]
                Ci = (C0_i[i] + stoichiometric_ratio * lim_conc * X) /
                dilution_factor
            else: # Inert (stoichiometric_coefficients[i] = 0)
                # For inerts, just dilution
                Ci = C0_i[i] / dilution_factor

        component_name = components[i]["component_name"]
        outlet_concentrations[component_name] = Ci
        concentration_values.append(Ci)

    return outlet_concentrations, k, r
```

Fonte: Autoria própria.

Para determinação do coeficiente de expansão volumétrico para reações gasosas, a Equação 4.5 deve ser utilizada.

$$\varepsilon = y_{A0} \cdot \frac{\sum n_P - \sum n_R}{|n_A|} \quad (4.5)$$

Dos quais,

- $\sum n_P$ – Somatório dos coeficientes estequiométricos dos produtos gasosos;
- $\sum n_R$ – Somatório dos coeficientes estequiométricos dos reagentes gasosos;
- y_{A0} – Fração molar do reagente limitante na alimentação;
- n_A – Coeficiente estequiométrico do reagente limitante.

$$\varphi = (1 + \epsilon X) \frac{P}{P_0} \frac{T_0}{T} \quad (4.6)$$

Figura 8 – Código da função `_calculate_dilution_factor()`

```
def _calculate_dilution_factor(self, components, stoichiometric_coefficients, limiting, y_A0):
    """Calculate the dilution factor based on the stoichiometric coefficients and yield."""
    has_gas_phase = any(c["state"] == "gaseous" for c in components)

    if has_gas_phase:
        mols_prod = sum(coef for coef in stoichiometric_coefficients if coef > 0)
        mols_reag = sum(abs(coef) for coef in stoichiometric_coefficients if coef < 0)
        ε = (mols_prod - mols_reag) / abs(stoichiometric_coefficients[limiting]) * y_A0
    else:
        ε = 0.0

    return ε
```

Fonte: Autoria própria.

Onde ϵ representa a variação de número de mols por mol de A reagido. O fator ψ agrega a expansão/contração estequiométrica (via ϵX) e as correções de pressão e temperatura em relação às condições iniciais (LEVENSPIEL, 2000). Essas relações estequiométricas permitem calcular as concentrações de saída de todos os componentes a partir da conversão X do reagente limitante, sendo a base para os balanços nos diferentes modelos de reatores.

4.2.2.1 Reator do Tipo CSTR

O reator do tipo CSTR é um tanque agitado operando em regime contínuo. Assume-se mistura completa no interior do tanque (composição uniforme em todo o volume). A conversão é desenvolvida integralmente dentro do volume do reator, dependendo do balanço entre a cinética de reação e o tempo de residência dos reagentes no tanque.

No sistema implementado, são suportados três casos de uso: (i) conversão desejada e cinética conhecidas (calcula-se o volume do reator necessário), (ii) volume disponível e cinética conhecidas (calcula-se a conversão atingida) e (iii) tempo de residência e cinética conhecidos (calcula-se a conversão atingida). As saídas calculadas pelo algoritmo incluem,

entre outros: conversão do reagente limitante X , vazão molar do reagente limitante na entrada F_{A0} vazão volumétrica total na saída Q_T , volume do reator V , taxa de reação r no interior, concentrações de saída $C_{I,saída}$, fator de diluição ψ , coeficiente de expansão ε e tempo de residência τ .

4.2.2.1.1 Conversão e Cinética (Cálculo do Volume)

Passos principais do cálculo:

- Identificação do reagente limitante (conforme descrito anteriormente);
- Cálculo das vazões molares de entrada pela Equação 4.7:

$$N_{i0} = Q_i \cdot C_{i0e} \quad (4.7)$$

para cada componente, bem como da vazão volumétrica total pela Equação 4.8:

$$Q_T = \sum Q_i \quad (4.8)$$

As concentrações iniciais (após mistura das correntes de alimentação) são dadas pela Equação 4.9:

$$C_{i0} = \frac{N_{i0}}{Q_T} \quad (4.9)$$

- Cálculo do fator de diluição total ψ com a conversão alvo X (Equação 4.6);
- Cálculo da taxa de reação r a partir da constante cinética k e das concentrações na saída (considerando X), supondo regime isotérmico (FOGLER, 2009);
- Aplicação do balanço no CSTR para o reagente limitante.

Para conversão X conhecida, a equação de *design* é dada por:

$$V = \frac{F_{A0} \cdot X}{-r_A} \quad (4.10)$$

Dos quais,

- F_{A0} – Vazão molar de A na entrada
- r - Taxa de consumo de A por unidade de volume no interior do reator

Cálculo do tempo de residência:

$$\tau = \frac{Q_T}{V} \quad (4.11)$$

Figura 9 – Código da função `_conversion_and_kinetics_in_cstr()` – Parte 1/2

```
def _conversion_and_kinetics_in_cstr(self, parameters):  
    """Calculates the volume of a CSTR based on conversion and kinetics."""  
    self._require_keys(parameters, ["conversion"])  
    self._validate_numeric(parameters, ["conversion"])  
  
    init_data = self._initialize_reactor(parameters)  
    X = parameters["conversion"] * self.ureg.dimensionless  
  
    # Unpack needed variables  
    components = init_data["components"]  
    stoichiometric_coefficients = init_data["stoichiometric_coefficients"]  
    reaction_rate_params = init_data["reaction_rate_params"]  
    limiting = init_data["limiting"]  
    F_A0 = init_data["F_A0"]  
    C0_i = init_data["C0_i"]  
    C_lim0 = init_data["C_lim0"]  
    Q_tot = init_data["Q_tot"]  
    epsilon = init_data["epsilon"]  
    var_operation_conditions = init_data["var_operation_conditions"]  
  
    # Combined factor: effect of volumetric variation and operating conditions  
    dilution_factor = (1 + (epsilon * X)) * var_operation_conditions  
  
    outlet_concentrations, k, r = self._calculate_concentration_and_rate(  
        components, stoichiometric_coefficients, reaction_rate_params,  
        limiting, C0_i, C_lim0, X, dilution_factor  
    )  
  
    ...
```

Fonte: Autoria própria.

Figura 10 – Código da função `_conversion_and_kinetics_in_cstr()` – Parte 2/2

```
def _conversion_and_kinetics_in_cstr(self, parameters):
    """Calculates the volume of a CSTR based on conversion and kinetics."""
    ...

    if any(C.magnitude < 0 for C in outlet_concentrations.values()):
        raise ValueError("Too high conversion – negative concentration
calculated.")

    # Check if r has the correct units (mol/m³/s)
    if r.units != self.ureg.Unit('mol/m³/s'):
        raise ValueError(f"Incorrect unit for reaction rate: {r.units}. Expected:
mol/m³/s")

    if r.magnitude == 0:
        raise ValueError("The reaction rate cannot be zero.")

    # Consumption rate of the limiting reagent
    rate_lim = abs(stoichiometric_coefficients[limiting]) * r # mol/m³/s

    # Calculate the reactor volume using V/FA0 = X/r
    V = F_A0 * X / rate_lim # m³

    residence_time = V / Q_tot

    # Return the volume with the unit in cubic meters
    return {
        "volume": V,
        "reaction_rate": r,
        "outlet_concentrations": outlet_concentrations,
        "dilution_factor": epsilon * self.ureg.dimensionless,
        "molar_rate_inlet(limitant)": F_A0,
        "flow_rate_outlet": Q_tot,
        "residence_time": residence_time,
        "conversion": X,
    }
```

Fonte: Autoria própria.

A fim de simplificar as lógicas, é usada a função `_initialize_reactor()`, que calcula as concentrações e vazões de cada componente na entrada, assim como o *epsilon*, fator de diluição, temperaturas e pressões iniciais e finais e o reagente limitante.

Figura 11 – Código da função `_initialize_reactor()` – Parte 1/2

```
def _initialize_reactor(self, parameters):
    """Standardizes the initialization of reactor calculations."""
    # Common keys required for all calculations
    self._require_keys(parameters, ["components", "stoichiometric_coefficients",
    "operation_conditions"])

    components = parameters["components"]
    stoichiometric_coefficients = parameters["stoichiometric_coefficients"]
    operation_conditions = parameters["operation_conditions"]

    # Validate component states
    last_component_state = components[0]["state"]
    for component in components:
        if last_component_state != component["state"]:
            raise ValueError("Use only liquid or only gaseous components")

    limiting = self.determine_limiting_reagent(parameters)

    # Initialize variables
    F0_i = []
    Q_tot = 0 * self.ureg.m**3 / self.ureg.s

    # Calculate inlet molar flow rates and total volumetric flow rate
    for c in components:
        flow_rate = c["flow_rate_inlet"] * self.ureg.m**3 / self.ureg.s
        conc = (c["molar_concentration_inlet"] * self.ureg.mol /
self.ureg.L).to(self.ureg.mol / self.ureg.m**3)
        F0_i.append(flow_rate * conc)
        Q_tot += flow_rate

    F_A0 = F0_i[limiting]
    C0_i = [F0 / Q_tot for F0 in F0_i]
    C_lim0 = C0_i[limiting]
    y_A0 = F_A0 / sum(F0_i)

    ...
```

Fonte: Autoria própria.

Figura 12 – Código da função `_initialize_reactor()` – Parte 2/2


```
def _initialize_reactor(self, parameters):
    """Standardizes the initialization of reactor calculations."""
    ...

    epsilon = self._calculate_dilution_factor(components,
stoichiometric_coefficients, limiting, y_A0)

    # Operating conditions
    T0 = operation_conditions["initial_temperature"] * self.ureg.K
    P0 = operation_conditions["initial_pressure"] * self.ureg.Pa
    T = operation_conditions["final_temperature"] * self.ureg.K
    P = operation_conditions["final_pressure"] * self.ureg.Pa

    var_operation_conditions = (P0 * T / (P * T0)).magnitude

    return {
        "components": components,
        "stoichiometric_coefficients": stoichiometric_coefficients,
        "reaction_rate_params": parameters["reaction_rate_params"],
        "operation_conditions": operation_conditions,
        "limiting": limiting,
        "F_A0": F_A0,
        "C0_i": C0_i,
        "C_lim0": C_lim0,
        "Q_tot": Q_tot,
        "epsilon": epsilon,
        "var_operation_conditions": var_operation_conditions,
        "T": T,
        "P": P,
        "T0": T0,
        "P0": P0
    }
```

Fonte: Autoria própria.

4.2.2.1.2 Volume e Cinética (Cálculo da Conversão)

Nesse caso, são conhecidos o volume do reator V e a cinética (expressão da taxa $r(C_i)$, incluindo k). Deseja-se determinar a conversão X atingida no CSTR. Isolando X na equação de *design* do CSTR obtém-se uma equação implícita. Define-se então uma função objetivo:

$$f(x) = \frac{|v_A| \cdot r_A \cdot V}{F_{A0}} - X \quad (4.12)$$

Então encontra-se sua raiz na faixa $0 < X < 1$, tipicamente utilizando o método de Brent, dada a monotonicidade esperada de $f(x)$. A raiz X que satisfaz $f(x) = 0$ corresponde à conversão no reator.

Figura 13 – Código da função `_volume_and_kinetics_in_cstr()` – Parte 1/2

```
def _volume_and_kinetics_in_cstr(self, parameters):
    """Calculates the conversion of a CSTR based on volume and kinetics."""
    self._require_keys(parameters, ["volume"])
    self._validate_numeric(parameters, ["volume"])

    init_data = self._initialize_reactor(parameters)
    V = parameters["volume"] * self.ureg.m**3

    # Unpack needed variables
    components = init_data["components"]
    stoichiometric_coefficients = init_data["stoichiometric_coefficients"]
    reaction_rate_params = init_data["reaction_rate_params"]
    limiting = init_data["limiting"]
    F_A0 = init_data["F_A0"]
    C0_i = init_data["C0_i"]
    C_lim0 = init_data["C_lim0"]
    Q_tot = init_data["Q_tot"]
    epsilon = init_data["epsilon"]
    var_operation_conditions = init_data["var_operation_conditions"]

    def objective(X_val):
        if not (0 < X_val < 1):
            raise ValueError("Conversion out of bounds")

        dilution_factor = (1 + (epsilon * X_val)) * var_operation_conditions

        outlet_concentrations, k, r = self._calculate_concentration_and_rate(
            components, stoichiometric_coefficients, reaction_rate_params,
            limiting, C0_i, C_lim0, X_val, dilution_factor
        )

    ...
```

Fonte: Autoria própria.

Figura 14 – Código da função `_volume_and_kinetics_in_cstr()` – Parte 2/2

```
def _volume_and_kinetics_in_cstr(self, parameters):
    ...

    if any(Ci.magnitude < 0 for Ci in outlet_concentrations.values()):
        raise ValueError("Ci is negative")

    rate_lim = abs(stoichiometric_coefficients[limiting]) * r
    X_calc = (rate_lim * V / F_A0).to_base_units().magnitude
    return X_calc - X_val

    result = root_scalar(objective, bracket=[1e-9, 1.0 - 1e-9], method='brentq')
    if not result.converged:
        raise ValueError("Failed to converge to a valid conversion value.")

    X = result.root * self.ureg.dimensionless

    dilution_factor = (1 + (epsilon * X)) * var_operation_conditions
    outlet_concentrations, k, r = self._calculate_concentration_and_rate(
        components, stoichiometric_coefficients, reaction_rate_params,
        limiting, C0_i, C_lim0, X, dilution_factor
    )

    return {
        "conversion": X,
        "reaction_rate": r,
        "outlet_concentrations": outlet_concentrations,
        "dilution_factor": epsilon * self.ureg.dimensionless,
        "molar_rate_inlet(limitant)": F_A0,
        "flow_rate_outlet": Q_tot,
        "residence_time": V / Q_tot,
        "volume": V,
    }
```

Fonte: Autoria própria.

4.2.2.1.3 Tempo de Residência e Cinética

Conhecido o tempo de residência τ , calcula-se inicialmente pela Equação 4.13:

$$V = Q_T \cdot \tau \quad (4.13)$$

Em seguida, o problema se reduz ao caso anterior: aplica-se a mesma função objetivo $f(X)$ da Equação 4.12 e resolve-se numericamente para X , obtendo a conversão atingida no CSTR com tempo de residência τ especificado.

Figura 15 – Código da função `_residence_time_and_kinetics_in_cstr()` – Parte 1/2

```

def _residence_time_and_kinetics_in_cstr(self, parameters):
    """Calculates the conversion of a CSTR based on residence time and
    kinetics."""
    self._require_keys(parameters, ["residence_time"])
    self._validate_numeric(parameters, ["residence_time"])

    init_data = self._initialize_reactor(parameters)
    residence_time = parameters["residence_time"] * self.ureg.s

    # Unpack needed variables
    components = init_data["components"]
    stoichiometric_coefficients = init_data["stoichiometric_coefficients"]
    reaction_rate_params = init_data["reaction_rate_params"]
    limiting = init_data["limiting"]
    F_A0 = init_data["F_A0"]
    C0_i = init_data["C0_i"]
    C_lim0 = init_data["C_lim0"]
    Q_tot = init_data["Q_tot"]
    epsilon = init_data["epsilon"]
    var_operation_conditions = init_data["var_operation_conditions"]

    # Reactor volume based on residence time
    V = Q_tot * residence_time # m³

    def objective(X_val):
        if not (0 < X_val < 1):
            raise ValueError("Conversion out of bounds")

        dilution_factor = (1 + (epsilon * X_val)) * var_operation_conditions

        outlet_concentrations, k, r = self._calculate_concentration_and_rate(
            components, stoichiometric_coefficients, reaction_rate_params,
            limiting, C0_i, C_lim0, X_val, dilution_factor
        )

    ...

```

Fonte: Autoria própria.

Figura 16 – Código da função `_residence_time_and_kinetics_in_cstr()` – Parte 2/2

```

def _residence_time_and_kinetics_in_cstr(self, parameters):
    ...

    if any(Ci.magnitude < 0 for Ci in outlet_concentrations.values()):
        raise ValueError("Ci is negative")

    rate_lim = abs(stoichiometric_coefficients[limiting]) * r
    X_calc = (rate_lim * V / F_A0).to_base_units().magnitude
    return X_calc - X_val

    result = root_scalar(objective, bracket=[1e-9, 1.0 - 1e-9], method='brentq')
    if not result.converged:
        raise ValueError("Failed to converge to a valid conversion value.")

    X = result.root * self.ureg.dimensionless

    dilution_factor = (1 + (epsilon * X)) * var_operation_conditions
    outlet_concentrations, k, r = self._calculate_concentration_and_rate(
        components, stoichiometric_coefficients, reaction_rate_params,
        limiting, C0_i, C_lim0, X, dilution_factor
    )

    return {
        "conversion": X,
        "molar_rate_inlet_(limitant)": F_A0,
        "flow_rate_outlet": Q_tot,
        "volume": V,
        "reaction_rate": r,
        "outlet_concentrations": outlet_concentrations,
        "dilution_factor_(1+e * P0*T/P*T0)": dilution_factor *
self.ureg.dimensionless,
        "residence_time": V / Q_tot,
        "dilution_factor": epsilon * self.ureg.dimensionless
    }

```

Fonte: Autoria própria.

4.2.2.2 Reator do Tipo PFR

Um reator tubular é caracterizado por escoamento pistonado, sem mistura axial significativa: cada elemento de fluido que entra percorre o reator como se fosse um reator batelada em movimento, evoluindo em conversão conforme o comprimento.

Para uma conversão alvo X , o volume requerido de um PFR ideal é dado pela integração da equação de *design* diferencial ao longo da conversão (LEVENSPIEL, 2000). Matematicamente:

$$V = (R + 1) \cdot F_{A0} \cdot \int_{\frac{R}{R+1}X}^X \frac{dX}{-r_A} \quad (4.14)$$

Tendo que R representa a razão de reagentes não limitantes inertes em relação ao limitante (se houver variação de vazão volumétrica) e o termo $(R + 1)$ acomoda a variação de vazão na integração. No código, essa integral é avaliada numericamente (por exemplo, via *scipy.integrate.quad*). Para o caso inverso, em que V é conhecido e se busca X , utiliza-se novamente um procedimento iterativo (método de Brent) aplicando a equação de *design* diferencial até encontrar a conversão cujo volume calculado corresponda ao especificado (FOGLER, 2009).

Considerando a similaridade estrutural e semântica com as lógicas de cálculo do CSTR, vale ressaltar no que diferem, sendo o cálculo para volume, retratado na figura abaixo.

Figura 17 – Código da função objetivo para o PFR

```
def objective(X_val):
    # Inner function to calculate rate at a specific conversion x
    def rate_at_x(x):
        dilution_factor = (1 + (epsilon * x)) * expansion_factor
        _, _, r = self._calculate_concentration_and_rate(
            components, stoichiometric_coefficients, reaction_rate_params,
            limiting, C0_i, C_lim0, x, dilution_factor
        )
        return abs(stoichiometric_coefficients[limiting]) * r.magnitude

    # Integrate dX/-rA from Xinlet to X_val
    lower_limit = (R / (R + 1) * X_val).magnitude
    upper_limit = X_val

    if upper_limit <= lower_limit:
        integral_val = 0
    else:
        integral_val, _ = quad(lambda x: 1 / rate_at_x(x), lower_limit, upper_limit)

    # V = (R+1) * FA0 * integral
    F_A0_mag = F_A0.to(self.ureg.mol / self.ureg.s).magnitude
    V_calc = (R + 1) * F_A0_mag * integral_val * self.ureg.m**3

    return (V_calc - V).to_base_units().magnitude
```

Fonte: Autoria própria.

4.2.3 Tubulações

Para um correto cálculo de perdas de carga e seleção de bombas, o *software* incorpora dados de catálogo de tubulações, rugosidades de materiais e comprimentos equivalentes de acessórios.

Rugosidade de materiais: A rugosidade absoluta do material interno do tubo influencia o fator de atrito em regime turbulento, através da rugosidade relativa ($\frac{\epsilon}{D}$). Materiais metálicos, plásticos ou com incrustações apresentam valores típicos de rugosidade tabelados (WHITE,

2018). Com o tempo de operação, depósitos e corrosão podem elevar a rugosidade efetiva, devendo-se considerar fatores de segurança.

Catálogo de diâmetros: São armazenadas dimensões padrão de tubulações (diâmetro externo, espessura e diâmetro interno resultante) para diferentes *schedules* (e.g., SCH 10, 40, 80) e materiais. Também podem ser incluídos dados de pressão máxima de trabalho e peso por metro. Isso permite que, dado um diâmetro calculado, o programa selecione o diâmetro nominal comercial mais próximo disponível.

Figura 18 – Especificações *hardcoded* para tubulações

```
class Piping:
    def __init__(self):
        """
        Contains piping specifications.
        Each number key is the nominal diameter in mm, which retrieve:
        ----external_diameter: mm
        ----thickness: mm
        ----weight: kg/m
        ----pressure: psi
        """

        self.ureg = UnitRegistry()

        self.data = {
            "dimensions":{
                "SCH10": {
                    6: {
                        "external_diameter": 10.30,
                        "thickness": 1.245,
                        "weight": 0.277,
                        "max_pressure": None
                    },
                    ...
                },
                "SCH40": {
                    ...
                },
                ...
            },
            "composition":{
                "Commercial steel":{
                    "roughness":0.06,
                    "roughness_coefficient": 135
                },
                ...
            },
            "fittings": {
                "180 degrees Return": {
                    "equivalentLength": 28
                },
                ...
            }
        }
```

Fonte: Autoria própria.

Coeficientes de perda em acessórios: Curvas, válvulas, expansões, contrações e conexões

diversas introduzem perdas de carga locais. No modelo implementado, cada acessório pode ser contabilizado por um comprimento equivalente L_{eq} (ou fator K de perda) adimensional expresso em unidades de diâmetro $L_{eq}D$). O comprimento equivalente total da tubulação é dado por $\sum L_{eq}$ de todos os acessórios multiplicado pelo diâmetro da tubulação, e somado ao comprimento linear. Esses dados (fatores K ou L_{eq} típicos para cada acessório padrão) foram compilados de literatura de engenharia de escoamento (PERRY; GREEN, 2007).

No cálculo de perdas de carga totais, o comprimento L da tubulação reta é acrescido do comprimento equivalente $\sum L_{eq}$ das singularidades presentes. Uma estimativa inadequada da rugosidade ou do efeito de acessórios pode produzir erros significativos na previsão de perda de carga, resultando em superdimensionamento ou subdimensionamento de bombas. Assim, o uso de dados acurados de catálogo e correlações de atrito é fundamental para a confiabilidade do dimensionamento. Exemplificando para o cálculo da perda de carga por Darcy-Weisbach utilizando acessórios na figura abaixo.

Figura 19 – Cálculo da perda de carga por Darcy-Weisbach com acessórios

```
def _equivalent_length(
    self, fittings: List[Dict[str, object]] | None, diameter_m: "pint.Quantity"
) -> "pint.Quantity":
    """Return the total equivalent length of all fittings."""
    if not fittings:
        return 0 * self.ureg.m

    total = 0 * self.ureg.m
    for f in fittings:
        spec = self.piping.fitting_specifications(f["fitting"])
        total += spec["specifications"]["equivalentLength"].magnitude * diameter_m * f["quantity"]
    return total

def head_loss(self, parameters: Dict[str, object]):
    ...
    if method == "Darcy-Weisbach":
        req = ["friction_factor", "velocity", "pipe_length", "diameter"]
        self._require_keys(parameters, req)
        self._validate_numeric(parameters, req)

        f = parameters["friction_factor"]
        L = parameters["pipe_length"] * self.ureg.m
        V = parameters["velocity"] * self.ureg.m / self.ureg.s
        D = parameters["diameter"] * self.ureg.mm
        D_m = D.to(self.ureg.m)

        Leq = self._equivalent_length(parameters.get("fittings"), D_m)

        hl = f * (L + Leq) * V**2 / (2 * D_m * self.g)
        if hl < 0:
            raise ValueError(f"Head loss cannot be negative, got {hl}.")
        return hl.to(self.ureg.m)
```

Fonte: Autoria própria.

4.2.4 Balanço de Massa

O balanço de massa aplica o princípio da conservação da massa a sistemas de processo: em regime estacionário, a massa que entra em um sistema deve ser igual à massa que sai, ajustada pela geração ou consumo devido a reações químicas (FELDER; ROUSSEAU, 2005). Para cada componente, a quantidade produzida ou consumida deve ser contabilizada segundo a estequiometria.

$$Acúmulo = Entradas - Saídas + Geração - Consumo \quad (4.15)$$

No estado estacionário sem reação, o acúmulo é zero, de modo que:

$$\sum \dot{m}_{entrada} = \sum \dot{m}_{saída} \quad (4.16)$$

Por componente i , incluindo reação química:

$$\sum \dot{m}_{entrada,i} + \dot{m}_{geração,i} = \sum \dot{m}_{saída} + \dot{m}_{consumo,i} \quad (4.17)$$

Dos quais.

- $\dot{m}$, sendo a vazão mássica

Em problemas industriais, frequentemente há reciclo e purgas envolvidos nos balanços. Nesses casos, para divisores de fluxo (*splitters*), define-se por exemplo:

$$F_R = f \cdot F_P \quad (4.18)$$

$$F_G = (1 - f) \cdot F_P \quad (4.19)$$

Dos quais,

- f , sendo a fração de reciclo
- F_G , sendo a vazão total do produto bruto
- F_R , sendo a vazão do reciclo
- F_P , sendo a vazão da purga

Esses parâmetros permitem fechar balanços em sistemas com recirculação e calcular a acumulação de inertes ou subprodutos. Na Figura 20, é apresentado o resultado gerado pelo *software* para um balanço de massa contendo reciclo e reação.

Figura 20 – Resultados do balanço de massa com reciclo e reação

Mass Balance Results				
Flow rates are in consistent units throughout. Compositions are mass fractions (when using mass flow units) or molar fractions (when using molar flow units).				
Process Metrics - Fresh Feed: 100.00 (mass or mol)/time - Product Flow: 50.00 (mass or mol)/time - Recycle Ratio: 0.75				
Fresh_Feed Flow Rate: 100.00 (mass or mol)/time Compositions (mass or molar fractions): A: 0.8000 B: 0.2000 C: 0.0000 D: 0.0000				
Reactor_Out Flow Rate: 125.00 (mass or mol)/time Compositions (mass or molar fractions): A: 0.1574 B: 0.0183 C: 0.6426 D: 0.1817				
Recycle Flow Rate: 75.00 (mass or mol)/time Compositions (mass or molar fractions): A: 0.1574 B: 0.0183 C: 0.6426 D: 0.1817				
Product Flow Rate: 50.00 (mass or mol)/time Compositions (mass or molar fractions): A: 0.1574 B: 0.0183 C: 0.6426 D: 0.1817				

Fonte: Autoria própria.

O balanço de massa fundamenta o dimensionamento de equipamentos como misturadores, separadores, reatores e redes de processo, além de embasar análises econômicas (minimizando perdas de matérias-primas) e de segurança (evitando acúmulo indesejado de compostos). Vale notar que temperatura e pressão influenciam o balanço apenas através de propriedades físicas (densidade, volumes específicos), ou mudanças de fase que afetem as vazões mássicas e molares de cada corrente (FELDER; ROUSSEAU, 2005).

4.2.5 Escoamento, Perdas de Carga e Diâmetros Hidráulicos

O comportamento do escoamento em dutos é governado, principalmente, pelo número de Reynolds, que permite classificar o regime como laminar, de transição ou turbulento. Variáveis-chave incluem o diâmetro hidráulico, a velocidade média, a viscosidade e a densidade do fluido, bem como a rugosidade relativa da parede. Perfis de velocidade, mecanismos de mistura e dissipação de energia por atrito dependem desse regime de escoamento. Para quantificar a perda de carga (queda de pressão) devida ao atrito no escoamento, emprega-se a equação universal de Darcy–Weisbach (WHITE, 2018).

No regime turbulento, o fator de atrito f depende de Re e de $\frac{\epsilon}{D}$ (rugosidade relativa), não possuindo solução analítica explícita – daí o uso de relações empíricas ou aproximadas. Já no regime laminar ($Re < 2300$), o perfil de velocidades é parabólico e f admite

expressão simples dada pela Equação 4.20:

$$f = \frac{64}{Re} \quad (4.20)$$

Em serviços de água a escoamento turbulento, a fórmula empírica de Hazen–Williams é comumente empregada como aproximação prática (PERRY; GREEN, 2007). Temperatura e pressão afetam densidade e viscosidade do fluido, modificando o valor de Re e, consequentemente, as perdas de carga.

Além disso, o material e condições da tubulação (nova, envelhecida) determinam a rugosidade efetiva. Todos esses efeitos impactam diretamente o balanço de energia em uma linha de fluxo e a potência requerida em bombas ou compressores (PERRY; GREEN, 2007; WHITE, 2018).

4.2.5.1 Número de Reynolds

O número de Reynolds (Re) é adimensional e relaciona as forças inerciais com as forças viscosas no escoamento dentro de um duto:

Usando viscosidade dinâmica μ :

$$Re = \frac{D \cdot V \cdot \rho}{\mu} \quad (4.21)$$

Usando viscosidade cinemática ν :

$$Re = \frac{D \cdot V}{\nu} \quad (4.22)$$

Onde D é o diâmetro hidráulico, ρ a densidade do fluido, V é a velocidade média e $\mu = \frac{\nu}{\rho}$ é a viscosidade cinemática. Tipicamente, $Re < 2300$ indica regime laminar, $2300 < Re < 4000$ transição, e $Re > 4000$ regime turbulento plenamente desenvolvido (WHITE, 2018).

Figura 21 – Cálculo do Reynolds – Parte 1/3



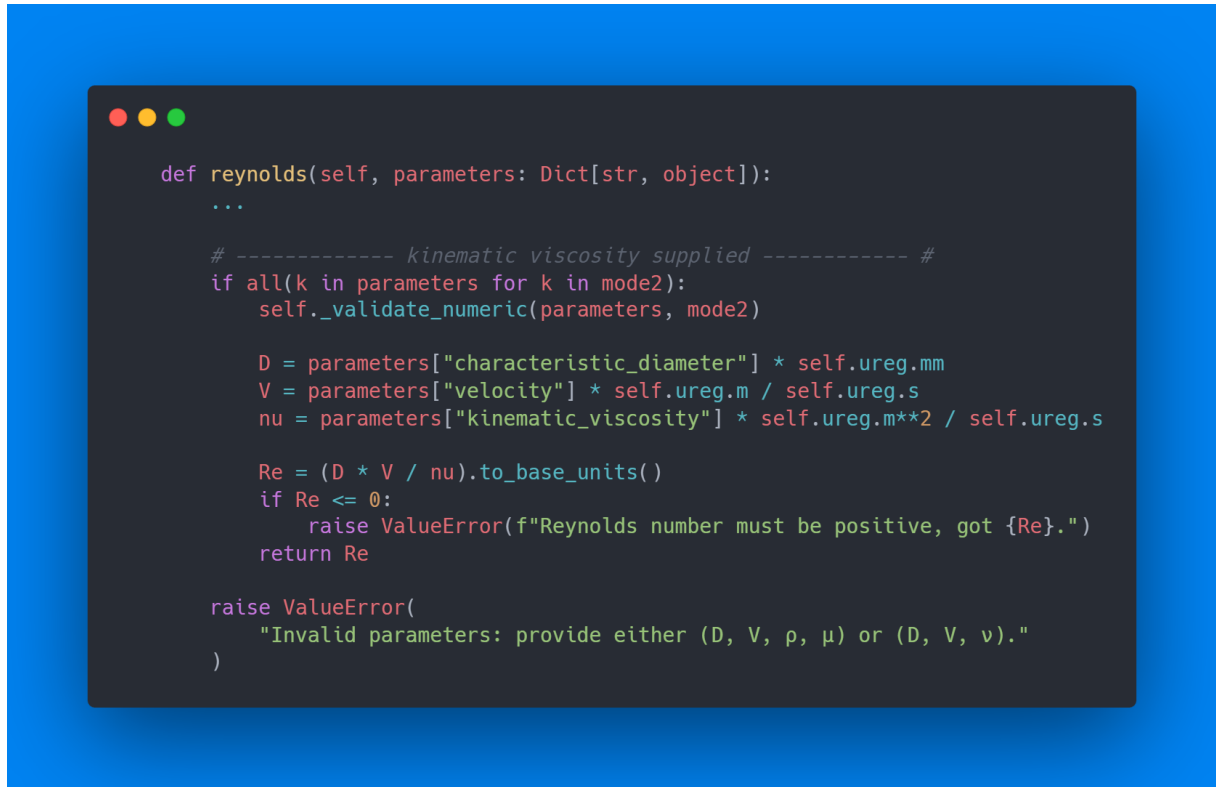
Fonte: Autoria própria.

Figura 22 – Cálculo do Reynolds – Parte 2/3

```
def reynolds(self, parameters: Dict[str, object]):  
    ...  
  
    # ----- dynamic viscosity supplied ----- #  
    if all(k in parameters for k in model):  
        self._validate_numeric(parameters, model)  
  
        D = parameters["characteristic_diameter"] * self.ureg.mm  
        V = parameters["velocity"] * self.ureg.m / self.ureg.s  
        rho = parameters["density"] * self.ureg.kg / self.ureg.m**3  
        mu = parameters["dynamic_viscosity"] * self.ureg.kg / (  
            self.ureg.m * self.ureg.s  
        )  
  
        Re = (D * rho * V / mu).to_base_units()  
        if Re <= 0:  
            raise ValueError(f"Reynolds number must be positive, got {Re}.")  
        return Re  
    ...
```

Fonte: Autoria própria.

Figura 23 – Cálculo do Reynolds – Parte 3/3



Fonte: Autoria própria.

4.2.5.2 Fator de Atrito

No regime laminar, o fator de atrito de Darcy pode ser obtido diretamente por $f = \frac{64}{Re}$. No regime turbulento, f satisfaz implicitamente a equação de Colebrook–White (1939) (COLEBROOK, 1939):

$$\frac{1}{\sqrt{f}} = -2 \log \left(\frac{\varepsilon}{3,7D_h} + \frac{2,51}{Re\sqrt{f}} \right) \quad (4.23)$$

Sendo ε a rugosidade absoluta. Para evitar a resolução iterativa, existem correlações explícitas aproximadas amplamente usadas, por exemplo:

Swamee–Jain (1976) (SWAMEE; JAIN, 1976):

$$f = \frac{0,25}{\left[\log \left(\frac{\varepsilon}{3,7D} + \frac{5,74}{Re^{0,9}} \right) \right]^2} \quad (4.24)$$

Haaland (1983) (HAALAND, 1983):

$$\frac{1}{\sqrt{f}} = -1,8 \log \left(\frac{\varepsilon/D}{3,7} + \frac{6,9}{Re} \right) \quad (4.25)$$

O *software* implementa as equações acima para cálculo de f conforme o regime e dados disponíveis (WHITE, 2018). A interface para a obtenção do fator de atrito e as opções disponibilizadas ao usuário estão exibidas na Figura 24.

Figura 24 – Interface de cálculo do fator de atrito

Friction Factor

Roughness Type
☒ Material Composition ☐ Custom Value

Material Composition
Commercial steel

Diameter Type
☒ Pipe Schedule ☐ Custom Value

Pipe Schedule
SCH40

Diameter (mm)
15 mm

Reynolds Number
100000

Method
ColebrookWhite

[Calculate Friction Factor](#) [Limpar campos](#)

Property	Value	Units
Diameter	0.0294801	dimensionless

Fonte: Autoria própria.

Figura 25 – Cálculo do fator de atrito – Parte 1/2

```
def friction_factor(self, parameters: Dict[str, object]):  
    """  
    Darcy friction factor.  
  
    Parameters  
    -----  
    parameters : dict  
        ``{"method": "ColebrookWhite" | "SwameeJain" | "Haaland",  
         "roughness": <mm>,  
         "diameter": <mm>,  
         "reynolds": <float>}```  
  
    Returns  
    -----  
    pint.Quantity  
        Darcy friction factor (dimensionless).  
    """  
    if "method" not in parameters:  
        return ["ColebrookWhite", "SwameeJain", "Haaland"]  
  
    req = ["roughness", "diameter", "reynolds", "method"]  
    self._require_keys(parameters, req)  
    self._validate_numeric(parameters, ["roughness", "diameter", "reynolds"])  
  
    eps = parameters["roughness"]  
    D = parameters["diameter"]  
    Re = float(parameters["reynolds"])  
    eps_over_D = eps / D # dimensionless ratio  
  
    method = parameters["method"]  
  
    ...
```

Fonte: Autoria própria.

Figura 26 – Cálculo do fator de atrito – Parte 2/2

```

def friction_factor(self, parameters: Dict[str, object]):
    ...

    # ----- Colebrook-White ----- #
    if method == "ColebrookWhite":

        def objective_x(x):
            # x represents 1/sqrt(f)
            return x + 2 * np.log10(eps_over_D / 3.71 + 2.51 * x / Re)

        try:
            res = root_scalar(objective_x, bracket=[0.1, 100], method='brentq')
            x_sol = res.root
            f_solution = 1 / (x_sol ** 2)
            return float(f_solution) * self.ureg.dimensionless
        except ValueError:
            # Fallback/Error handling
            raise ValueError("Colebrook equation solver failed to converge.")

    # ----- Swamee-Jain ----- #
    if method == "SwameeJain":
        f = 0.25 / (
            np.log10(eps_over_D / 3.7 + 5.74 / (Re**0.9))
        ) ** 2
        return float(f) * self.ureg.dimensionless

    # ----- Haaland ----- #
    if method == "Haaland":
        inv_sqrt_f = -1.8 * np.log10(
            (eps_over_D / 3.7) ** 1.11 + 6.9 / Re
        )
        f = 1 / inv_sqrt_f**2
        return float(f) * self.ureg.dimensionless

    raise ValueError("Invalid friction factor method.")

```

Fonte: Autoria própria.

4.2.5.3 Perda de Carga – Darcy-Weisbach e Hazen-Williams

A perda de carga distribuída h_f em uma tubulação de comprimento L é calculada pela equação de Darcy-Weisbach:

$$h_f = f \cdot \frac{(L + \sum L_{eq})}{D} \frac{V^2}{2g} \quad (4.26)$$

Sendo $\sum L_{eq}$ é o comprimento equivalente total dos acessórios (em unidades de comprimento), e g a aceleração da gravidade. Essa fórmula é universal e independe do fluido, desde que f seja adequadamente determinado (PERRY; GREEN, 2007).

Para escoamento de água em tubulações de diâmetro moderado (tipicamente $D > 50mm$) e regime turbulento plenamente estabelecido, a fórmula empírica de Hazen–Williams pode ser empregada como simplificação:

$$h_f = \frac{4,73L}{C^{1,852} \cdot D^{4,87}} Q^{1,852} \quad (4.27)$$

Sendo Q é vazão volumétrica e C é o coeficiente de Hazen–Williams do material (adimensional, tabelado para tubulações novas de ferro fundido, aço, PVC etc.). Apesar de não possuir fundamentação teórica geral, essa equação fornece boa precisão para água a 20°C em tubulações de transporte, e é consagrada no projeto de redes de água devido à sua simplicidade ([PERRY; GREEN, 2007](#); [WHITE, 2018](#)).

Figura 27 – Cálculo da perda de carga – Parte 1/3

```

def head_loss(self, parameters: Dict[str, object]):
    """
    Head loss by Darcy-Weisbach or Hazen-Williams.

    Parameters
    -----
    parameters : dict
        **Darcy-Weisbach**

        ``{"method": "Darcy-Weisbach",
         "friction_factor": <float>,
         "pipe_length": <m>,
         "diameter": <mm>,
         "velocity": <m/s>,
         "fittings": [{"fitting": <str>, "quantity": <int>}, ...]}``

        **Hazen-Williams**

        ``{"method": "Hazen-Williams",
         "flow_rate": <m³/s>,
         "roughness_coefficient": <float>,
         "pipe_length": <m>,
         "diameter": <mm>,
         "fittings": [...]} ``

    Returns
    -----
    pint.Quantity
        Head loss (m).
    """
    if "method" not in parameters:
        return ["Darcy-Weisbach", "Hazen-Williams"]

    method = parameters["method"]

    ...

```

Fonte: Autoria própria.

Figura 28 – Cálculo da perda de carga – Parte 2/3

```
def head_loss(self, parameters: Dict[str, object]):  
    ...  
  
    # ----- Darcy-Weisbach ----- #  
    if method == "Darcy-Weisbach":  
        req = ["friction_factor", "velocity", "pipe_length", "diameter"]  
        self._require_keys(parameters, req)  
        self._validate_numeric(parameters, req)  
  
        f = parameters["friction_factor"]  
        L = parameters["pipe_length"] * self.ureg.m  
        V = parameters["velocity"] * self.ureg.m / self.ureg.s  
        D = parameters["diameter"] * self.ureg.mm  
        D_m = D.to(self.ureg.m)  
  
        Leq = self._equivalent_length(parameters.get("fittings"), D_m)  
  
        hl = f * (L + Leq) * V**2 / (2 * D_m * self.g)  
        if hl < 0:  
            raise ValueError(f"Head loss cannot be negative, got {hl}.")  
        return hl.to(self.ureg.m)  
  
    ...
```

Fonte: Autoria própria.

Figura 29 – Cálculo da perda de carga – Parte 3/3

```

def head_loss(self, parameters: Dict[str, object]):
    ...

    # ----- Hazen-Williams ----- #
    if method == "Hazen-Williams":
        req = ["flow_rate", "roughness_coefficient", "diameter", "pipe_length"]
        self._require_keys(parameters, req)
        self._validate_numeric(parameters, req)

        Q = parameters["flow_rate"] * self.ureg.m**3 / self.ureg.s
        C = parameters["roughness_coefficient"]
        L = parameters["pipe_length"] * self.ureg.m
        D = parameters["diameter"] * self.ureg.mm

        if D.magnitude < 50:
            raise ValueError("For Hazen-Williams the diameter must exceed 50
mm.")

        D_m = D.to(self.ureg.m)
        Leq = self._equivalent_length(parameters.get("fittings"), D_m)

        hl = (
            10.67 * (L + Leq) * Q**1.85 / (C**1.85 * D_m**4.87)
        ) * self.ureg.s**1.85 / self.ureg.m**0.68
        if hl <= 0:
            raise ValueError(f"Head loss must be positive, got {hl}.")
        return hl

    raise ValueError('Invalid method. Use "Darcy-Weisbach" or "Hazen-Williams".')

```

Fonte: Autoria própria.

4.2.5.4 Altura Manométrica entre Dois Pontos

A forma estendida da equação de Bernoulli é utilizada para avaliar as variações de carga (energia por unidade de peso do fluido) entre dois pontos de uma linha de escoamento, incluindo perdas:

$$H = \frac{P_2 - P_1}{\rho g} + \frac{V_2^2 - V_1^2}{2g} + z_2 - z_1 \quad (4.28)$$

Sendo H a altura manométrica adicionada (se positiva) ou removida (se negativa) entre o ponto 1 e 2. Nessa equação, $\frac{P_2 - P_1}{\rho g}$ representa a variação de pressão estática convertida em altura de coluna de fluido, $z_2 - z_1$ é a variação de cota geométrica, o termo de velocidades representa a variação de energia cinética, e h_f engloba todas as perdas de carga (distribuídas e localizadas) entre os pontos (WHITE, 2018).

Essa equação é fundamental para avaliar a necessidade de bombeamento: uma bomba deve fornecer uma H suficiente para superar as perdas e elevar o fluido da condição 1 até

2; inversamente, uma turbomáquina geradora extrairá essa energia do fluido.

Figura 30 – Cálculo do *head* – Parte 1/2

```
def head(self, parameters: Dict[str, object]):  
    """  
    Calculate the manometric head between two points in a pipe system.  
  
    Parameters  
    -----  
    parameters : dict  
        ``{"pressure1": <Pa>,  
          "pressure2": <Pa>,  
          "elevation1": <m>,  
          "elevation2": <m>,  
          "velocity1": <m/s>,  
          "velocity2": <m/s>,  
          "specific_mass": <kg/m3>,  
          "friction_factor": <m>}``  
  
    Returns  
    -----  
    pint.Quantity  
        Manometric head (m).  
    """  
    req = [  
        "pressure1",  
        "pressure2",  
        "elevation1",  
        "elevation2",  
        "velocity1",  
        "velocity2",  
        "specific_mass",  
        "friction_factor"  
    ]  
    self._require_keys(parameters, req)  
    self._validate_numeric(parameters, req)  
    ...
```

Fonte: Autoria própria.

Figura 31 – Cálculo do *head* – Parte 2/2

```

def head(self, parameters: Dict[str, object]):
    ...

    # Get parameters with proper units
    p1 = parameters["pressure1"] * self.ureg.Pa
    p2 = parameters["pressure2"] * self.ureg.Pa
    z1 = parameters["elevation1"] * self.ureg.m
    z2 = parameters["elevation2"] * self.ureg.m
    v1 = parameters["velocity1"] * self.ureg.m / self.ureg.s
    v2 = parameters["velocity2"] * self.ureg.m / self.ureg.s
    specific_mass = parameters["specific_mass"] * self.ureg.kg / self.ureg.m**3
    friction_factor = parameters["friction_factor"] * self.ureg.m

    # Equation terms
    pressure_term = (p2 - p1) / (specific_mass * self.g)
    elevation_term = (z2 - z1)
    velocity_term = (v2**2 - v1**2) / (2 * self.g)

    # Head calculation: (p2-p1)/specific_mass*g + (z2-z1) + (v2-v1)/(2g) +
    # friction_factor
    head = pressure_term + elevation_term + velocity_term + friction_factor

    # Return in meters
    return head.to(self.ureg.m)

```

Fonte: Autoria própria.

4.2.5.5 NPSH Disponível

Para evitar cavitação em bombas, calcula-se o NPSH disponível na sucção, que deve exceder o NPSH requerido pelo fabricante. A expressão para o NPSH disponível, em termos de altura de líquido, é:

$$NPSH_{disp} = \frac{P_{suc} + P_{atm} - p_{vap}}{\rho g} + \frac{V_{suc}^2}{2g} + (z_{suc} - z_0) - h_{suc} \quad (4.29)$$

Sendo P_{suc} é a pressão manométrica na linha de sucção da bomba, P_{atm} a pressão atmosférica, z_{suc} a elevação do ponto de sucção em relação a um P_{atm} de referência, v_{suc} a velocidade do fluido na sucção, h_{suc} as perdas de carga na linha de sucção, e p_{vap} a pressão de vapor do líquido na temperatura de bombeamento (PERRY; GREEN, 2007).

O cálculo de NPSH disponível permite verificar se há margem de segurança sobre o NPSH requerido da bomba (fornecido pelo fabricante para aquela vazão), garantindo que o líquido não vaporize na entrada da bomba – o que causaria cavitação, danos e redução de desempenho.

Figura 32 – Cálculo do *NPSH* – Parte 1/2

```
def npsht_available(self, parameters: Dict[str, object]):  
    """  
    Calculate available NPSH.  
  
    Parameters  
    -----  
    parameters : dict  
        ``{"manometric_pressure": <kgf/cm2>,  
         "atmospheric_pressure": <kgf/cm2>,  
         "vapor_pressure": <kgf/cm2>,  
         "specific_mass": <kg/m3>,  
         "friction_factor": <m>,  
         "pump_inlet_velocity": <m/s>,  
         "gauge_elevation": <m>}```  
  
    Returns  
    -----  
    pint.Quantity  
        Available NPSH (m).  
    """  
    req = [  
        "manometric_pressure",  
        "atmospheric_pressure",  
        "vapor_pressure",  
        "specific_mass",  
        "friction_factor",  
        "pump_inlet_velocity"  
    ]  
    self._require_keys(parameters, req)  
    self._validate_numeric(parameters, req)  
    ...
```

Fonte: Autoria própria.

Figura 33 – Cálculo do *NPSH* – Parte 2/2

```

def npsh_available(self, parameters: Dict[str, object]):
    ...

    # Get parameters with proper units
    Ps = parameters["manometric_pressure"] * self.ureg.kgf / self.ureg.cm**2
    Patm = parameters["atmospheric_pressure"] * self.ureg.kgf / self.ureg.cm**2
    Pv = parameters["vapor_pressure"] * self.ureg.kgf / self.ureg.cm**2
    specific_mass = parameters["specific_mass"] * self.ureg.kg / self.ureg.m**3
    friction_factor = parameters["friction_factor"] * self.ureg.m
    pump_inlet_velocity = parameters["pump_inlet_velocity"] * self.ureg.m /
self.ureg.s
    height_term = parameters["gauge_elevation"] * self.ureg.m

    # Convert pressures
    # Convert to N/m²
    Ps_Pa = Ps.to(self.ureg.kgf / self.ureg.cm**2).to(self.ureg.Pa)
    Patm_Pa = Patm.to(self.ureg.kgf / self.ureg.cm**2).to(self.ureg.Pa)
    Pv_Pa = Pv.to(self.ureg.kgf / self.ureg.cm**2).to(self.ureg.Pa)

    # Equation terms
    pressure_term = (Ps_Pa + Patm_Pa) / (specific_mass * self.g)
    vapor_pressure_term = Pv_Pa / (specific_mass * self.g)
    velocity_term = (pump_inlet_velocity**2) / (2 * self.g)

    # New equation: (Pmanometric+Patm)/gamma + V²/(2g) + h - friction_factor -
    Pvap/gamma
    npsh = pressure_term + height_term + velocity_term - friction_factor -
    vapor_pressure_term

    # Return in meters
    return npsh.to(self.ureg.m)

```

Fonte: Autoria própria.

4.2.5.6 Diâmetro Hidráulico para Geometrias Diversas

O conceito de diâmetro hidráulico generaliza o cálculo do número de Reynolds e do fator de atrito para seções transversais não circulares. O diâmetro hidráulico é definido como:

$$D_h = \frac{4A}{P} \quad (4.30)$$

onde A é a área da seção transversal de escoamento e P o perímetro molhado (contorno em contato com o fluido). Assim:

Para dutos circulares, conforme a Equação 4.31:

$$D_h = D \quad (4.31)$$

(recupera o diâmetro interno). Para canais ou abertos, conforme a Equação 4.32:

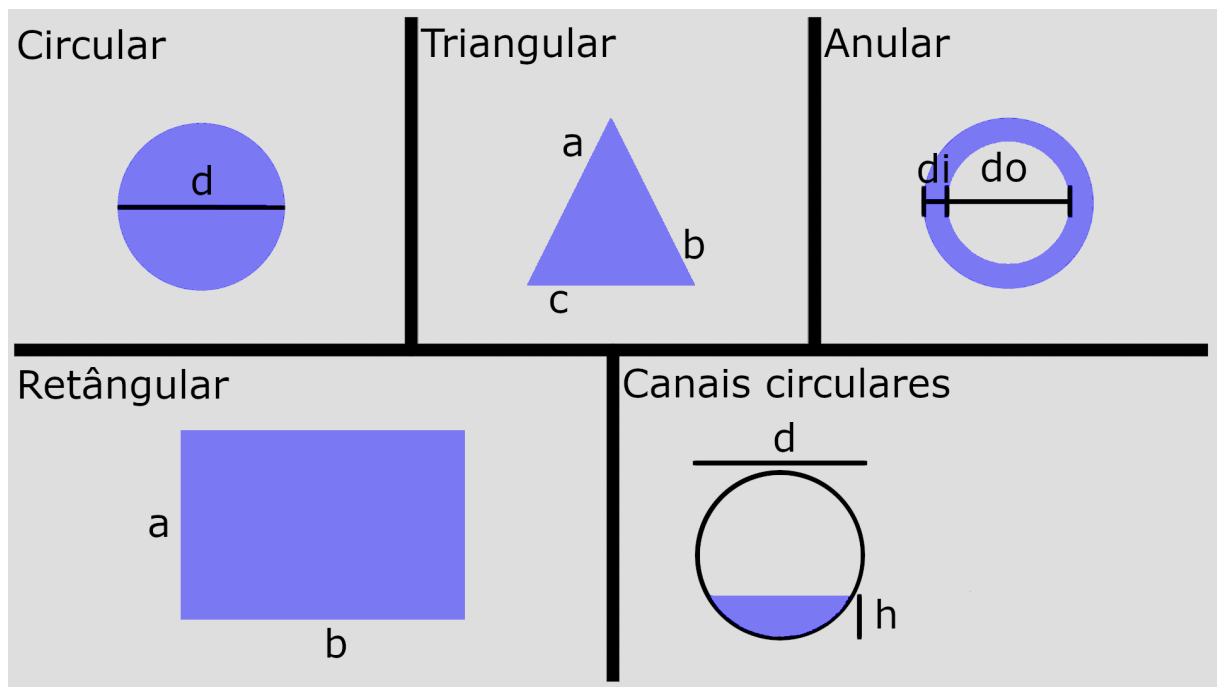
$$D_h = \frac{4ab}{2(a+b)} \quad (4.32)$$

onde a e b são os lados do retângulo. Para anulares (espaço entre dois tubos concêntricos), conforme a Equação 4.33:

$$D_h = D_o - D_i \quad (4.33)$$

(diferença entre diâmetro externo interno e diâmetro externo do interno). Para seções triangulares (por ex., condutos abertos triangulares), calcula-se A e P conforme a geometria e aplica-se $4A/P$. Para geometrias mais complexas (ex.: calhas semicirculares), também se aplica $4A/P$ considerando o perímetro molhado adequadamente.

Figura 34 – Tipos de dutos de escoamento



Fonte: Autoria própria.

O programa permite ao usuário informar A e P para cálculo de D_h em situações especiais, garantindo flexibilidade para tratar geometrias arbitrárias (WHITE, 2018).

Figura 35 – Exemplo da identificação e critérios de diâmetro hidráulico

```

def hydraulic_diameter(self, parameters: Dict[str, object]):
    """
    Calculate the hydraulic diameter for different geometric shapes.

    Parameters
    -----
    parameters : dict
        ``{"shape": <str>, ...shape-specific parameters...}``
        **Circular**
        ``{"shape": "circular", "diameter": <mm>}``
        **Rectangular**
        ``{"shape": "rectangular", "width": <mm>, "height": <mm>}``
        **Annular**
        ``{"shape": "annular", "outer_diameter": <mm>, "inner_diameter": <mm>}``
        **Triangular**
        ``{"shape": "triangular", "side_a": <mm>, "side_b": <mm>, "side_c": <mm>}``
        **Circular Cap**
        ``{"shape": "circularCap", "diameter": <mm>, "height": <mm>}``

    Returns
    -----
    pint.Quantity
        Hydraulic diameter (mm).
    """
    if "shape" not in parameters:
        return ["circular", "rectangular", "annular", "triangular",
                "circularCap"]

    shape = parameters["shape"]

    if shape == "circular": ...
    elif shape == "rectangular": ...
    elif shape == "annular": ...
    elif shape == "triangular": ...
    elif shape == "circularCap": ...

```

Fonte: Autoria própria.

4.2.6 Propriedades Termofísicas

Propriedades termofísicas – como densidade, viscosidade, calor específico, condutividade térmica, entalpia e entropia – são insumos essenciais em praticamente todos os cálculos de processos. Em misturas, a predição de propriedades requer modelos mais complexos do que simples somas ponderadas, considerando interações moleculares e desvios da idealidade. Tais propriedades alimentam balanços de massa e energia, o dimensionamento térmico de trocadores, o cálculo de Reynolds e de fatores de atrito, o desempenho de separações, entre outros. Temperatura e pressão são variáveis determinantes: utilizam-se correlações empíricas (por exemplo, equação de Antoine para pressão de vapor) ou equações de estado (gás ideal, Peng-Robinson, SRK) conforme o sistema. Erros na estimativa de viscosidade

ou densidade afetam o cálculo de Reynolds, fator de atrito e potência de bombeamento; imprecisões em calores específicos ou entalpias propagam-se em balanços de energia e projeto de utilidades.

Figura 36 – Obtenção de propriedades pela biblioteca *CoolProp*

```

def get_property(self, fluid, property_name, temperature, pressure):
    """
    Returns a specific property for a fluid at given temperature and pressure

    Parameters:
    -----
    fluid : str
        Name of the fluid
    property_name : str
        Name of the property to retrieve (CoolProp format)
    temperature : float
        Temperature in K
    pressure : float
        Pressure in Pa

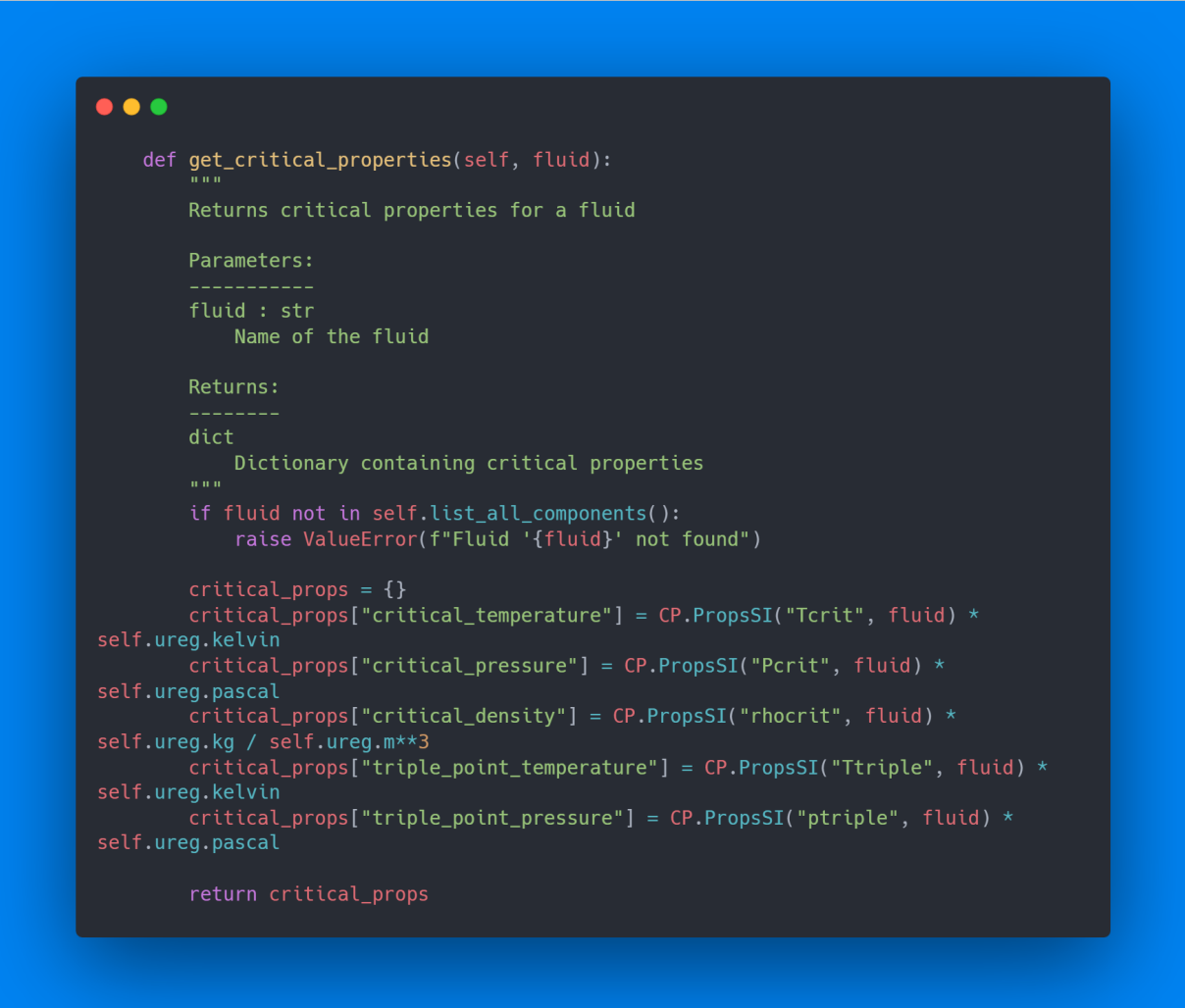
    Returns:
    -----
    float with units
        The requested property with appropriate units
    """
    if fluid not in self.list_all_components():
        raise ValueError(f"Fluid '{fluid}' not found")

    # Map of properties to their units
    units_map = {
        "D": self.ureg.kg / self.ureg.m**3,          # Density
        "C": self.ureg.joule / (self.ureg.kg * self.ureg.kelvin), # Specific
        "V": self.ureg.pascal * self.ureg.second,    # Viscosity
        "L": self.ureg.watt / (self.ureg.meter * self.ureg.kelvin), #
        "H": self.ureg.joule / self.ureg.kg,          # Enthalpy
        "S": self.ureg.joule / (self.ureg.kg * self.ureg.kelvin), # Entropy
        "M": self.ureg.kg / self.ureg.mol,            # Molecular weight
        "I": self.ureg.newton / self.ureg.meter,       # Surface tension
        "P": self.ureg.pascal,                        # Pressure
        "T": self.ureg.kelvin,                        # Temperature
        "T_bubble": self.ureg.kelvin,                 # Bubble point temperature
        "T_dew": self.ureg.kelvin,                   # Dew point temperature
        "P_bubble": self.ureg.pascal,                 # Bubble point pressure
        "P_dew": self.ureg.pascal                    # Dew point pressure
    }

    # Check if the fluid is pure (not a mixture)
    # For bubble and dew points, we need a mixture
    if property_name in ["T_bubble", "T_dew", "P_bubble", "P_dew"]:
        # For pure fluids, return the saturation temperature/pressure
        try:
            if property_name == "T_bubble" or property_name == "T_dew":
                # Saturation temperature at given pressure
                value = CP.PropsSI("T", "P", pressure, "Q", 0, fluid)
            elif property_name == "P_bubble" or property_name == "P_dew":
                # Saturation pressure at given temperature
                value = CP.PropsSI("P", "T", temperature, "Q", 0, fluid)
            except Exception as e:
                return None
        else:
            # For other properties, use standard method
            try:
                value = CP.PropsSI(property_name, "T", temperature, "P", pressure,
fluid)
            except Exception as e:
                raise ValueError(f"CoolProp error for property '{property_name}' of
fluid '{fluid}' at T={temperature}K, P={pressure}Pa: {str(e)}")

    if property_name in units_map:
        return value * units_map[property_name]
    else:
        return value # Return without units if not in the map

```

Figura 37 – Obtenção de propriedades críticas pela biblioteca *CoolProp*

```
def get_critical_properties(self, fluid):  
    """  
    Returns critical properties for a fluid  
  
    Parameters:  
    -----  
    fluid : str  
        Name of the fluid  
  
    Returns:  
    -----  
    dict  
        Dictionary containing critical properties  
    """  
    if fluid not in self.list_all_components():  
        raise ValueError(f"Fluid '{fluid}' not found")  
  
    critical_props = {}  
    critical_props["critical_temperature"] = CP.PropsSI("Tcrit", fluid) *  
self.ureg.kelvin  
    critical_props["critical_pressure"] = CP.PropsSI("Pcrit", fluid) *  
self.ureg.pascal  
    critical_props["critical_density"] = CP.PropsSI("rhocrit", fluid) *  
self.ureg.kg / self.ureg.m**3  
    critical_props["triple_point_temperature"] = CP.PropsSI("Ttriple", fluid) *  
self.ureg.kelvin  
    critical_props["triple_point_pressure"] = CP.PropsSI("ptriple", fluid) *  
self.ureg.pascal  
  
    return critical_props
```

Fonte: Autoria própria.

No *software* desenvolvido, um módulo de propriedades utiliza a biblioteca *CoolProp* para recuperar propriedades de fluidos puros (e misturas) a partir de dados de entrada de T e P . São obtidas propriedades como densidade, viscosidade, calor específico (c_p e c_v), condutividade térmica, entalpia, entropia, massa molar e tensão superficial, além de propriedades críticas e pontos de saturação (bolha/orvalho) quando aplicável. A visualização desse cálculo de propriedades, com foco no resultado para uma mistura, é apresentada na Figura 38.

Figura 38 – Interface de cálculo para as propriedades de uma mistura

Mixture Properties

Temperature (K): 300 Pressure (Pa): 101325

Fluid Fractions

Water 0,5

Methanol 0,5

Properties to Calculate

Density Specific heat

Leave empty to calculate all available properties

Calculate Mixture Properties

Mixture Properties

Fluid Composition:

Water: 0.5 Methanol: 0.5

Property	Value	Units
Density	753.601	kilogram / meter ** 3
Specific Heat	3474.14	joule / kelvin / kilogram

Fonte: Autoria própria.

Para misturas, a *API* do *CoolProp* permite montar *strings* do tipo "*HEOS::Componente1[x1]&Componente2[x2]*" e retornar as propriedades solicitadas (BELL et al., 2014). Todas as grandezas retornam já associadas a unidades do SI (via integração com a biblioteca *Pint*), garantindo consistência nos cálculos.

Figura 39 – Obtenção de propriedades de misturas pela biblioteca *CoolProp* – Parte 1/2

```

def get_mixture_properties(self, fluid_fractions, temperature, pressure,
properties=None):
    """
    Returns properties for a mixture of fluids at given temperature and pressure

    Parameters:
    -----
    fluid_fractions : dict
        Dictionary with fluid names as keys and their mass fractions as values
        Example: {"Water": 0.7, "Ethanol": 0.3} for a mixture of 70% water and
30% ethanol
    temperature : float
        Temperature in K
    pressure : float
        Pressure in Pa
    properties : list, optional
        List of property keys to retrieve. If None, returns all available
properties.

    Returns:
    -----
    dict
        Dictionary containing the requested properties for the mixture
    """
    # Validate inputs
    for fluid in fluid_fractions.keys():
        if fluid not in self.list_all_components():
            raise ValueError(f"Fluid '{fluid}' not found")

    # Check that fractions sum to approximately 1
    total_fraction = sum(fluid_fractions.values())
    if not 0.99 <= total_fraction <= 1.01:
        raise ValueError(f"Sum of fluid fractions should be 1.0, got
{total_fraction}")

    # Create the mixture string for CoolProp
    # Format: "HEOS::fluid1[fraction1]&fluid2[fraction2]&..."
    mixture_string = "HEOS::"
    for i, (fluid, fraction) in enumerate(fluid_fractions.items()):
        if i > 0:
            mixture_string += "&"
            mixture_string += f"{fluid}[{fraction}]"

    # Get properties
    result = {}

    # Define which properties to retrieve
    if properties is None:
        properties = ["D", "C", "V", "L", "H", "S", "M", "I", "P", "T"]

    ...

```

Fonte: Autoria própria.

Figura 40 – Obtenção de propriedades de misturas pela biblioteca *CoolProp* – Parte 2/2

```

def get_mixture_properties(self, fluid_fractions, temperature, pressure,
properties=None):
    ...

    # Map of property keys to result dictionary keys
    property_key_map = {
        "D": "density",
        "C": "specific_heat",
        "V": "viscosity",
        "L": "thermal_conductivity",
        "H": "enthalpy",
        "S": "entropy",
        "M": "molecular_weight",
        "I": "surface_tension",
        "P": "pressure",
        "T": "temperature"
    }

    # Map of properties to their units (same as in get_property method)
    units_map = {
        "D": self.ureg.kg / self.ureg.m**3,          # Density
        "C": self.ureg.joule / (self.ureg.kg * self.ureg.kelvin), # Specific
        "V": self.ureg.pascal * self.ureg.second,    # Viscosity
        "L": self.ureg.watt / (self.ureg.meter * self.ureg.kelvin), #
        "H": self.ureg.joule / self.ureg.kg,          # Enthalpy
        "S": self.ureg.joule / (self.ureg.kg * self.ureg.kelvin), # Entropy
        "M": self.ureg.kg / self.ureg.mol,            # Molecular weight
        "I": self.ureg.newton / self.ureg.meter,       # Surface tension
        "P": self.ureg.pascal,                        # Pressure
        "T": self.ureg.kelvin                         # Temperature
    }

    # Retrieve each property
    for prop in properties:
        try:
            # For mixtures or other properties
            value = CP.PropsSI(prop, "T", temperature, "P", pressure,
mixture_string)

            if prop in units_map:
                result[property_key_map.get(prop, prop)] = value *
units_map[prop]
            else:
                result[property_key_map.get(prop, prop)] = value
        except Exception as e:
            raise ValueError(f"CoolProp mixture error for property '{prop}':
{str(e)}")

    return result

```

Fonte: Autoria própria.

O uso de uma biblioteca de propriedades de qualidade referência como o *CoolProp* assegura que o *software* tenha um *backend* termodinâmico rigoroso e abrangente, reforçando o caráter didático ao permitir explorar diferentes cenários com fidelidade física.

4.3 API

Com o objetivo de ampliar os casos de uso da ferramenta desenvolvida, a integração entre as camadas de *frontend* e *backend* é realizada por meio de uma API pública, permitindo que sistemas externos também possam consumir as funcionalidades disponibilizadas. Essa abordagem favorece a interoperabilidade do *software* e possibilita sua aplicação em diferentes contextos computacionais, como simulações, automações de processos e integração com outras plataformas utilizadas na engenharia química.

Para o consumo da API, a documentação encontra-se disponível no *endpoint* `/docs` do *backend*. Essa documentação foi desenvolvida utilizando o *framework* *FastAPI*, que possibilita a geração automática e padronizada da interface de documentação por meio da especificação *OpenAPI*, facilitando a compreensão, o teste e a integração dos *endpoints* disponibilizados.

4.4 Validação de Cálculos e Especificação de Retornos

Dentre as etapas fundamentais no desenvolvimento de *softwares*, a fase de testagem apresenta papel de grande relevância, uma vez que possibilita a verificação sistemática da conformidade dos resultados obtidos com aqueles previstos teoricamente. No contexto deste trabalho, essa etapa é essencial para garantir a confiabilidade dos cálculos e das rotinas computacionais empregadas em aplicações de Engenharia Química.

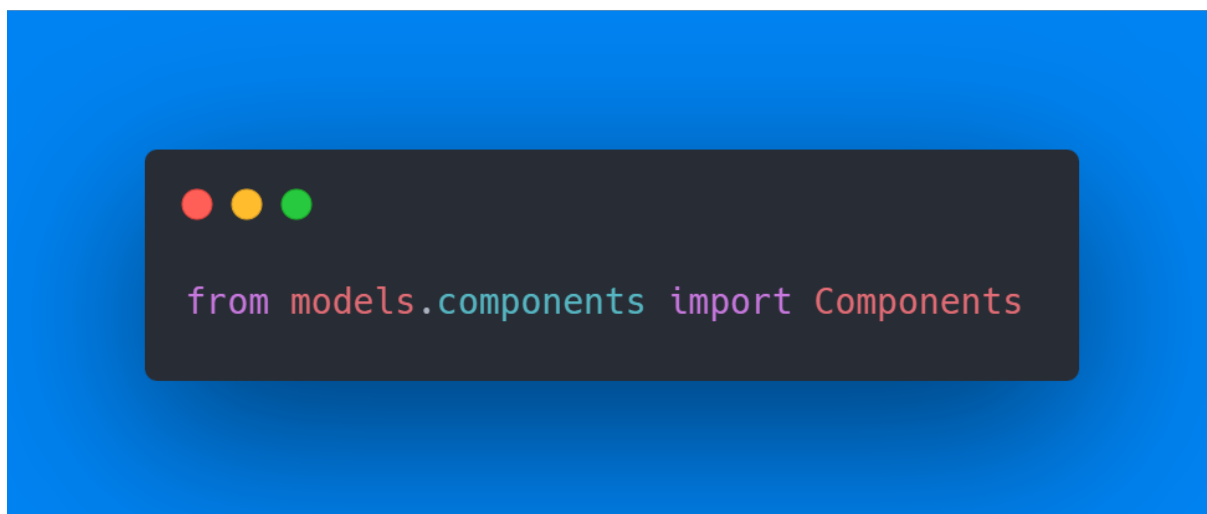
Uma abordagem eficiente para a realização desses testes consiste na utilização da biblioteca *pytest*, que permite a implementação de testes automatizados desenvolvidos para assegurar o correto funcionamento dos módulos do sistema. Por meio dessa ferramenta, é possível validar, de forma reprodutível e padronizada, os retornos das funções responsáveis pelos cálculos de propriedades, balanços e demais operações recorrentes da área.

Com o objetivo de detalhar o funcionamento do *software* desenvolvido, as funcionalidades de cada módulo são descritas a seguir, acompanhadas de evidências da interface *frontend* e de instruções para o consumo direto das rotinas por meio da linguagem *Python*, permitindo tanto a utilização via interface gráfica quanto a integração do sistema com outros projetos e aplicações computacionais.

4.4.1 Componentes

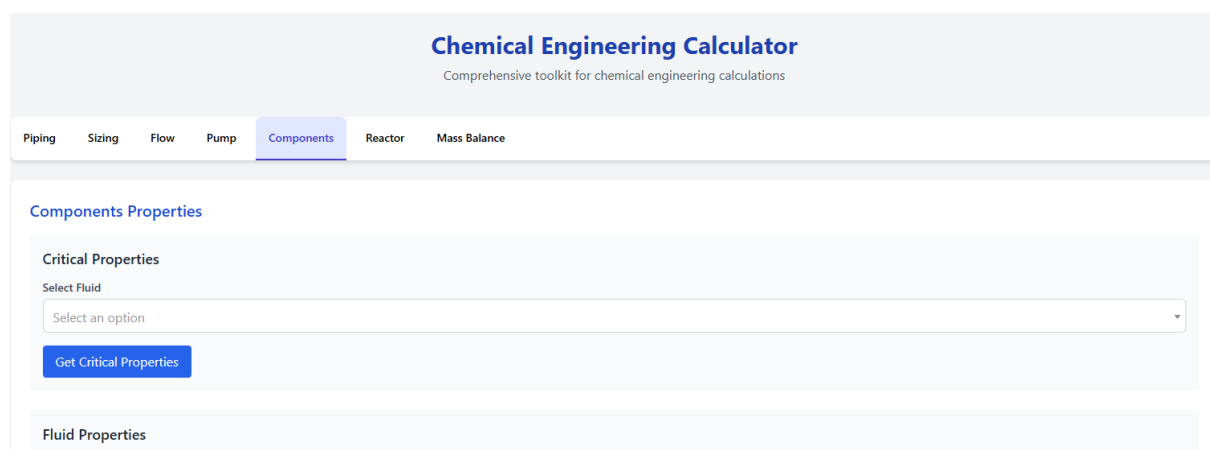
Para iniciar a biblioteca local com o módulo de componentes, o código da figura abaixo deve ser inserido, assegurando que o *Python* consiga identificar as funções dentro da classe *Components* contida no arquivo `./models/componentes.py`.

Tendo que o passo a passo para consumir via API ou *Python* o projeto seja demasiado técnico, será apenas brevemente comentado abaixo o modo de se fazer, depois havendo maior realce apenas aos retornos e comentários correlatos.

Figura 41 – Inicialização da biblioteca de componentes via *Python*

Fonte: Autoria própria.

No *frontend*, basta acessar a aba “*Components*”, vide imagem abaixo, e *scrollar* para baixo para visualizar as funcionalidades.

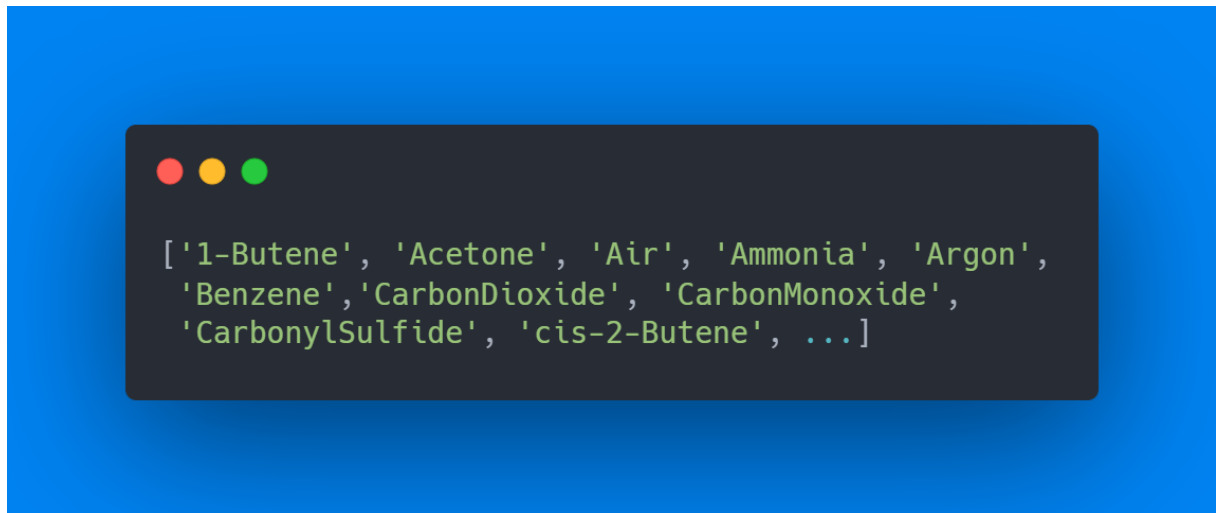
Figura 42 – Inicialização da biblioteca de componentes via *frontend*

Fonte: Autoria própria.

4.4.1.1 Listar Componentes

A função `list_all_components()` interage diretamente com a biblioteca termodinâmica *CoolProp* para listar todas as substâncias puras e pseudo-puras disponíveis para cálculo. O teste retornou um total de 124 fluidos cadastrados, abrangendo desde hidrocarbonetos simples e refrigerantes até fluidos inorgânicos como água e amônia. Esta listagem valida a capacidade do sistema em operar com uma vasta gama de componentes químicos industriais.

Figura 43 – Listagem parcial de componentes disponíveis na biblioteca termodinâmica



Fonte: Autoria própria.

4.4.1.2 Valor de Propriedade

Ao solicitar as propriedades termofísicas para o componente "Water" (Água) nas condições de $300K$ e $101325Pa$ ($1 atm$), o sistema retornou valores consistentes com a literatura. A massa específica calculada foi de $996,56 \text{ Kg}/m^3$ e a viscosidade de $8,54 \cdot 10^{-4} \text{ Pa} \cdot s$. Nota-se que o retorno inclui explicitamente as unidades físicas (utilizando a biblioteca *pint*), garantindo a consistência dimensional dos cálculos subsequentes.

Figura 44 – Propriedades termofísicas da água calculadas a 300K e 101,3kPa.

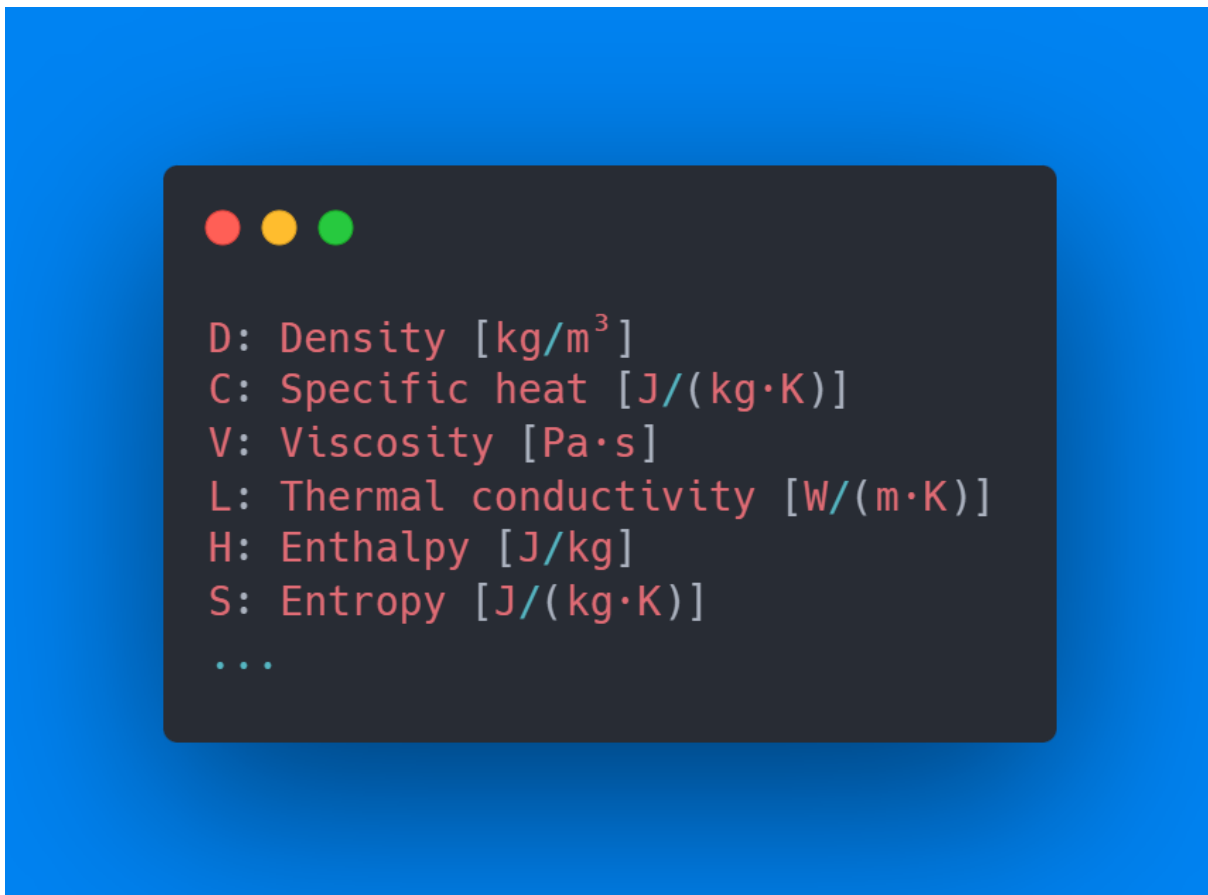


Fonte: Autoria própria.

4.4.1.3 Nome de Propriedades

A função `get_property_names()` expõe o mapeamento interno entre as chaves de solicitação da API (como 'D', 'H', 'V') e suas respectivas descrições físicas e unidades no SI. Esta estrutura organizacional permite que o usuário identifique rapidamente quais variáveis termodinâmicas podem ser requisitadas ao modelo, facilitando a configuração de simulações de balanço de massa e energia.

Figura 45 – Mapeamento de chaves de propriedades e suas unidades padronizadas.

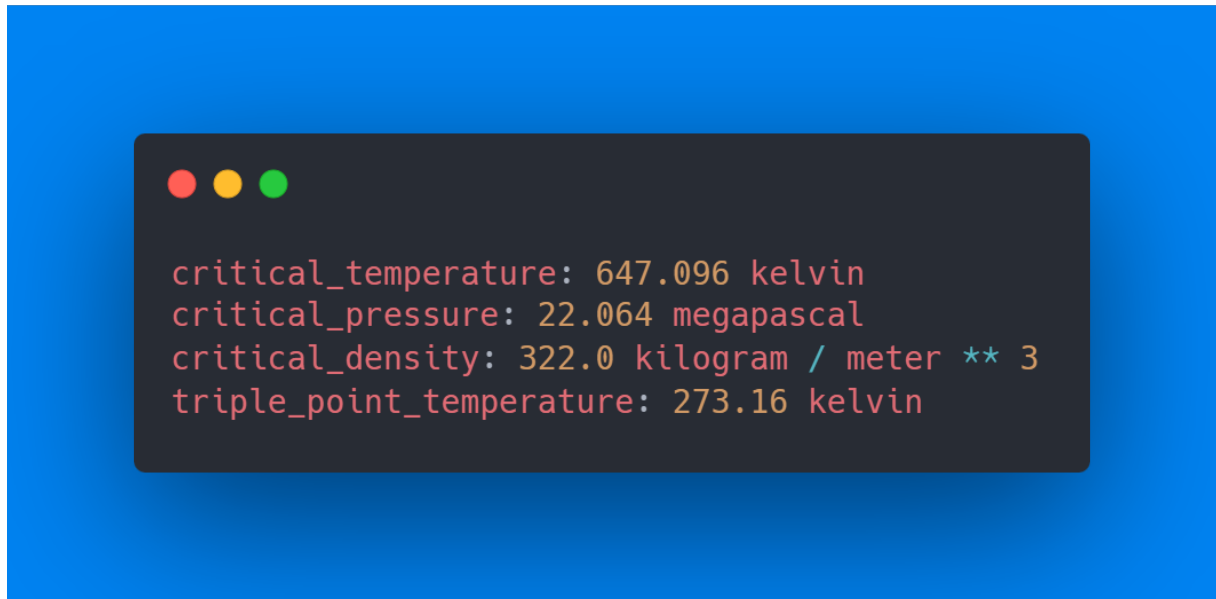


Fonte: Autoria própria.

4.4.1.4 Propriedades Críticas

A consulta das propriedades críticas para a água retornou uma temperatura crítica (T_c) de 647,1 K e uma pressão crítica (P_c) de 22,06 MPa. Estes parâmetros fixos são fundamentais para o cálculo de propriedades reduzidas e para a verificação dos limites de fase da substância nas equações de estado. O sistema também forneceu os dados do ponto triplo, expandindo a utilidade da ferramenta para análises de equilíbrio de fases.

Figura 46 – Propriedades críticas e do ponto triplo calculadas para a água.



Fonte: Autoria própria.

4.4.1.5 Opções de Propriedades de Misturas

Para sistemas multicomponentes, a função `get_property_mixture_names()` delimita o subconjunto de propriedades que podem ser calculadas. Diferentemente das substâncias puras, observa-se que certas propriedades de transporte específicas ou tensão superficial podem não estar disponíveis dependendo do modelo de mistura (*HEOS*) empregado. A listagem confirma a disponibilidade das variáveis de estado essenciais (entalpia, entropia, densidade) necessárias para o dimensionamento de reatores e tubulações.

Figura 47 – Propriedades termodinâmicas disponíveis para cálculo em misturas.

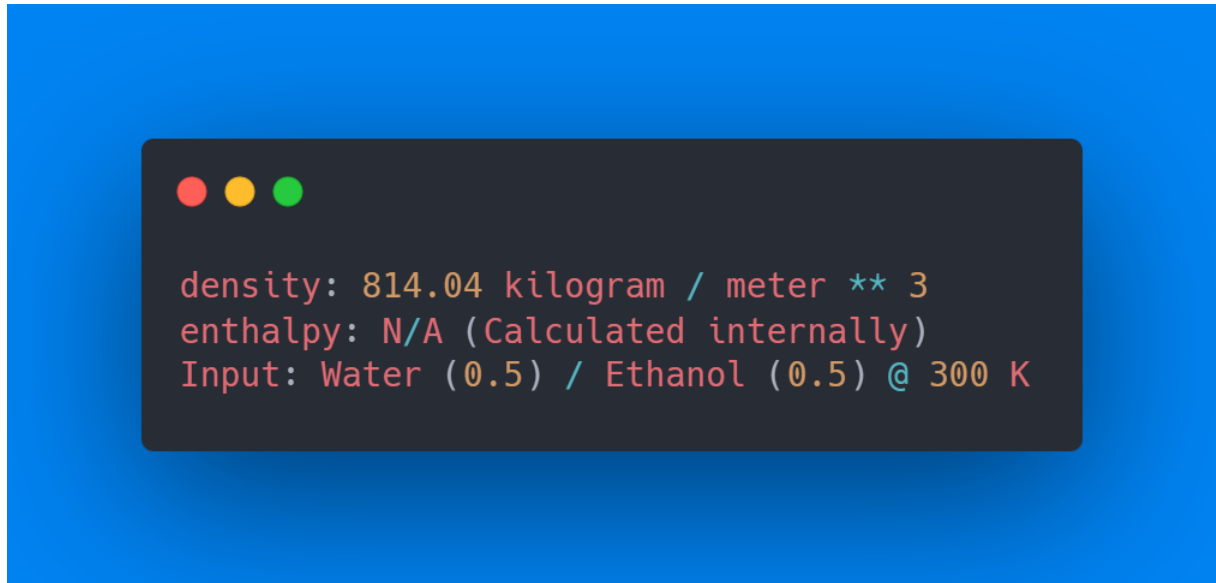


Fonte: Autoria própria.

4.4.1.6 Valor de Propriedades de Misturas

A simulação de uma mistura binária equimolar de Água (50%) e Etanol (50%) a $300K$ e $1atm$ resultou em uma densidade de mistura de $814,04\text{ kg/m}^3$. O código utiliza internamente a regra de mistura baseada na energia livre de Helmholtz (HEOS) do *CoolProp* para ponderar as interações moleculares entre os componentes. Este valor intermediário entre a densidade da água pura (997 kg/m^3) e do etanol puro (789 kg/m^3) demonstra a eficácia do algoritmo em prever o comportamento não-ideal da solução.

Figura 48 – Propriedades termodinâmicas disponíveis para cálculo em misturas.



Fonte: Autoria própria.

4.4.2 Hidráulicos

4.4.2.1 Perda de Carga

Para a validação do cálculo de perda de carga, utilizou-se o método de Darcy-Weisbach, aplicável a fluxos incompressíveis em qualquer regime de escoamento. A equação base para o cálculo é a Equação 4.26.

Onde a perda de carga localizada ($\sum h_{local}$) é calculada através do comprimento equivalente (L_{eq}) dos acessórios:

$$L_{total} = L + \sum(L_{eq} \cdot D) \quad (4.34)$$

Teste A: Tubulação Reta (Sem Acessórios)

- **Dados:** $L = 100m$, $D = 50mm(0,05m)$, $V = 2,0m/s$, $f = 0,02$
- **Cálculo Manual:**

$$h_f = 0,02 \cdot \frac{100}{0,05} \cdot \frac{2^2}{2 \cdot 9,80665} = 40 \cdot 0,2039 = 8,1577 \text{ m} \quad (4.35)$$

- **Software:** 8,1577 m
- **Status:** [MATCH] (Desvio: 0,00%)

Teste B: Com Acessórios (2 Curvas 90º Raio Longo)

- **Dados Adicionais:** 2 Cotovelos ($L_{eq}/D = 16$).

- **Comprimento Equivalente:** $L_{eq} = 2 \cdot (16 \cdot 0,05) = 1,6m$
- **Comprimento Total:** $101,6m$
- **Cálculo Manual:**

$$h_f = 0,02 \cdot \frac{101,6}{0,05} \cdot \frac{2^2}{2 \cdot 9,80665} = 8,2882 \text{ m} \quad (4.36)$$

- **Software:** $8,2882 \text{ m}$
- **Status:** [MATCH] (Desvio: 0,00%)

4.4.2.2 Reynolds

O Número de Reynolds (Re) foi validado utilizando as duas formas de entrada suportadas pela API: viscosidade dinâmica (μ) e cinemática (ν).

Método 1: Viscosidade Dinâmica (Equação 4.21)

- **Dados:** $\rho = 998kg/m^3, V = 2,0m/s, D = 0,05m, \mu = 1,002 \cdot 10^{-3}Pa \cdot s$
- **Cálculo Manual:** $Re = \frac{998 \cdot 2 \cdot 0,05}{0,001002} = 99.600,8$
- **Software:** $99.600,8$
- **Status:** [MATCH] (Desvio: 0,00%)

Método 2: Viscosidade Cinemática (Equação 4.22)

- **Dados:** $V = 2,0m/s, D = 0,05m, \nu = 1,004 \cdot 10^{-6}m^2/s$
- **Cálculo Manual:** $Re = \frac{2 \cdot 0,05}{1,004 \cdot 10^{-6}} = 99.601,6$
- **Software:** $99.601,6$
- **Status:** [MATCH] (Desvio: 0,00%)

4.4.2.3 Fator de Atrito

O fator de atrito de Darcy (f) foi validado comparando métodos iterativos e explícitos.

A. Consistência de Colebrook-White (COLEBROOK, 1939) Equação implícita (padrão, definida na Equação 4.23): Verificou-se a convergência da solução numérica da API comparando o Lado Esquerdo (LHS) e Direito (RHS) da equação.

- **Dados:** $\varepsilon = 0,045mm, D = 50mm, Re = 100.000$
- **Software** (f): $0,021614$
- **Check:** LHS = 6,802; RHS = 6,802

- **Status:** [CONVERGÊNCIA CONFIRMADA]

B. Equação de Swamee-Jain (Explícita) (**SWAMEE; JAIN, 1976**) (ver Equação 4.24)

- **Cálculo Manual:** $f = 0,02199$
- **Software:** 0,02199
- **Status:** [MATCH]

C. Equação de Haaland (Explícita) (**HAALAND, 1983**) (ver Equação 4.25)

- **Cálculo Manual:** $f = 0,02161$
- **Software:** 0,02161
- **Status:** [MATCH]

4.4.2.4 Diâmetro Real

A função de seleção de diâmetro comercial verifica o próximo diâmetro nominal disponível numa tabela de *Schedule*.

- **Entrada:** Diâmetro Calculado = 45,0 mm. Schedule = SCH40.
- **Lógica Manual:** Consultar tabela SCH40. Diâmetros disponíveis: ..., 32, 40, 50, 65... O próximo maior que 45 é 50.
- **Software:** 50,0 mm
- **Status:** [MATCH]

4.4.2.5 Diâmetro Calculado

O dimensionamento preliminar baseia-se na equação da continuidade para tubos circulares.

(ver Equação 4.1)

- **Dados:** $Q = 0,005 m^3/s$, $V = 1,5 m/s$
- **Cálculo Manual:** $D = \sqrt{\frac{4 \cdot 0,005}{\pi \cdot 1,5}} = \sqrt{0,004244} = 0,06515 m$ (65,15 mm)
- **Software:** 65,15 mm
- **Status:** [MATCH] (Desvio: 0,00%)

4.4.2.6 NPSH Disponível

O cálculo do *Net Positive Suction Head* disponível ($NPSH_a$) utiliza a equação de Bernoulli aplicada à sucção da bomba:

Utilizando a Equação 4.29

- **Dados:** $P_{man} = 0,5 \text{ kgf/cm}^2$, $P_{atm} = 1 \text{ atm}$, $P_{vap} = 0,02 \text{ kgf/cm}^2$, $\gamma = 9806 \text{ N/m}^3$, $h_f = 1 \text{ m}$, $\Delta Z = 2 \text{ m}$
- **Termos de Pressão (m.c.a):** $H_{press} = \frac{(0,5+1,033) \cdot 98.066,5}{9.806,6} = 15,33 \text{ m}$, $H_{vap} = 0,2 \text{ m}$
- **Cálculo Manual:** $15,33 + 2,0 + 0,11 - 1,0 - 0,2 = 16,24 \text{ m}$
- **Software:** $16,24 \text{ m}$
- **Status:** [MATCH] (Desvio: $< 0,01\%$)

4.4.2.7 Head

A carga manométrica total da bomba (H_{man}) é calculada pela diferença de energia entre sucção e descarga:

Utilizando a Equação 4.28

- **Dados:** $\Delta P = 101.325 \text{ Pa}$, $\Delta Z = 10 \text{ m}$, $V_1 = 1 \text{ m/s}$, $V_2 = 2 \text{ m/s}$, $h_f = 5 \text{ m}$
- **Cálculo Manual:** $\frac{101.325}{9806} + 10 + \frac{4-1}{19,6} + 5 = 10,33 + 10 + 0,15 + 5 = 25,48 \text{ m}$
- **Software:** $25,48 \text{ m}$
- **Status:** [MATCH] (Desvio: $0,00\%$)

4.4.2.8 Diâmetro Hidráulico

O cálculo do diâmetro hidráulico (D_h) foi validado para diferentes geometrias, fundamental para cálculos de perda de carga em dutos não circulares.

A relação geral é definida na Equação 4.30.

1. Circular

- **Dados:** $D = 50 \text{ mm}$
- **Manual:** $D_h = D = 50 \text{ mm}$
- **Software:** 50 mm [MATCH]

2. Retangular

- **Dados:** $L = 40mm, H = 30mm$
- **Manual:** $A = 1200, P = 140 \rightarrow D_h = \frac{4800}{140} = 34,29 \text{ mm}$
- **Software:** 34,29mm [MATCH]

3. Anular (Tubo Concêntrico)

- **Dados:** $D_{ext} = 100mm, D_{int} = 50mm$
- **Manual:** $D_h = D_{ext} - D_{int} = 50mm$
- **Software:** 50mm [MATCH]

4. Triangular

- **Dados:** Lados 3, 4, 5 (Triângulo Retângulo). $A = 6, P = 12$.
- **Manual:** $D_h = \frac{4 \cdot 6}{12} = 2,0mm$
- **Software:** 2,0mm [MATCH]

5. Canal Circular (Meia Seção)

- **Dados:** $D = 100mm$, Nível $H = 50mm$ (Metade).
- **Manual:** Para meia seção cheia, $A = \pi R^2/2, P = \pi R$. Logo $D_h = \frac{4(\pi R^2/2)}{\pi R} = 2R = D = 100mm$.
- **Software:** 100,0mm [MATCH]

4.4.3 Balanços de Massa

4.4.3.1 Misturador Ideal

A validação do balanço de massa foi realizada considerando um misturador ideal (Mixer) alimentado por duas correntes puras.

Definição do Caso Teste:

- **Corrente 1 (S1):** 100 kg/h de Água pura ($z_{\text{água}} = 1, 0$).
- **Corrente 2 (S2):** 100 kg/h de Etanol puro ($z_{\text{etanol}} = 1, 0$).
- **Operação:** Mistura adiabática sem reação química ($S1 + S2 \rightarrow S3$).

Cálculo Manual:

1. Balanço Global:

$$F_{S3} = F_{S1} + F_{S2} = 100 + 100 = 200 \text{ kg/h} \quad (4.37)$$

2. Balanço por Componente (Água):

$$F_{S3} \cdot z_{S3,agua} = F_{S1} \cdot z_{S1,agua} + F_{S2} \cdot z_{S2,agua} \quad (4.38)$$

$$200 \cdot z_{S3,agua} = 100 \cdot 1,0 + 100 \cdot 0,0 \implies z_{S3,agua} = 0,5 \quad (4.39)$$

Comparação com o Software: O código de validação foi executado conforme a Figura 49.

Figura 49 – Código de validação do Balanço de Massa (Misturador)

```
demo_mass_balance.py

import sys
import os

# Add parent directory to path to allow imports
sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), '..')))

from models.mass_balance import MassBalance
from demo.utils import print_header, print_result

def demo_mass_balance():
    print_header("Validação Balanço de Massa - Misturador")

    # Definição do Problema: Mistura de Água e Etanol
    # Corrente 1: 100 kg/h (Água Pura)
    # Corrente 2: 100 kg/h (Etanol Puro)
    # Misturador -> Corrente 3

    mb = MassBalance(components=["Água", "Etanol"])

    # 1. Adicionar Correntes
    # Stream 1: Entrada, 100 kg/h, 100% Água
    mb.add_stream("S1", ["Água", "Etanol"], 1, flow_rate=100.0, compositions={"Água": 1.0, "Etanol":
0.0})

    # Stream 2: Entrada, 100 kg/h, 100% Etanol
    mb.add_stream("S2", ["Água", "Etanol"], 1, flow_rate=100.0, compositions={"Água": 0.0, "Etanol":
1.0})

    # Stream 3: Saída
    mb.add_stream("S3", ["Água", "Etanol"], -1)

    print(">>> Dados de Entrada:")
    print("S1: F=100 kg/h, z=[Água: 1.0, Etanol: 0.0]")
    print("S2: F=100 kg/h, z=[Água: 0.0, Etanol: 1.0]")

    # 2. Solver
    results = mb.get_results()

    # 3. Resultados
    print("\n>>> Resultados Computacionais:")
    s3 = results["S3"]
    print(f"S3 Flow Rate: {s3['flow_rate']} kg/h")
    print(f"S3 Compositions: {s3['compositions']}")

    # Validação
    valid, msg = mb.validate_results(results)
    if valid:
        print("\n[VALIDAÇÃO] O balanço de massa fechou corretamente.")
    else:
        print(f"\n[ERRO] Falha na validação: {msg}")

if __name__ == "__main__":
    demo_mass_balance()
```

Tabela 3 – Validação Balanço de Massa (Misturador)

Parâmetro	Cálculo Manual	<i>Software</i>	Erro (%)
Vazão <i>S3</i> (<i>kg/h</i>)	200,0	200,0	0,00
Fração Água <i>S3</i>	0,50	0,50	0,00
Fração Etanol <i>S3</i>	0,50	0,50	0,00

Fonte: Autoria própria.

4.4.3.2 Reator de Conversão

Neste caso, avalia-se a geração e consumo de espécies químicas em um sistema reacional simples.

- **Reação:** $A \rightarrow B$ (Estequiometria 1:1).
- **Alimentação:** 100 *mol/s* de A puro.
- **Conversão** (X_A): 40%.

Cálculo Manual: Pela estequiometria, para cada mol de A consumido, gera-se 1 mol de B.

$$F_{A,out} = F_{A,in}(1 - X_A) = 100(1 - 0,40) = 60 \text{ mol/s} \quad (4.40)$$

$$F_{B,out} = F_{A,in}X_A = 100(0,40) = 40 \text{ mol/s} \quad (4.41)$$

A vazão molar total permanece constante ($60 + 40 = 100$), resultando nas frações molares:

$$y_A = \frac{60}{100} = 0,6; \quad y_B = \frac{40}{100} = 0,4 \quad (4.42)$$

Validação: O software retornou: $F_{Product} = 100 \text{ mol/s}$, com composição $z_A = 0,6$ e $z_B = 0,4$, confirmando a conservação de massa e estequiometria.

4.4.3.3 Divisor de Corrente (Splitter)

Simula-se uma operação de separação física onde uma corrente é dividida em duas (Reciclo e Produto) sem alteração de composição, típica em loops de reciclo.

- **Entrada no Divisor:** 100 *kg/h* (mistura 50% A, 50% B).
- **Razão de Divisão (Reciclo):** 0,2 ou 20%.

Cálculo Manual:

$$F_{Reciclo} = 0,2 \cdot 100 = 20 \text{ kg/h} \quad (4.43)$$

$$F_{Produto} = (1 - 0,2) \cdot 100 = 80 \text{ kg/h} \quad (4.44)$$

A composição deve permanecer inalterada: $z_A = 0,5$; $z_B = 0,5$ em ambas as saídas.

Validação: Conforme apresentado na saída expandida do software (Figura 50), os valores de vazão foram exatos (20 e 80 kg/h) e a composição manteve-se constante, validando a lógica do divisor.

Figura 50 – Resultados expandidos da validação de Balanço de Massa (Misturador, Reator e Splitter)

```

import sys
import os

# Add parent directory to path to allow imports
sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), '..')))

from models.mass_balance import MassBalance
from demo.utils import print_header, print_result

def demo_mixer():
    print_header("1. Validação Misturador")
    # Caso: Mistura de Água (S1) e Etanol (S2)
    mb = MassBalance(components=["Água", "Etanol"])
    mb.add_stream("S1", ["Água", "Etanol"], 1, flow_rate=100.0, compositions={"Água": 1.0, "Etanol":
0.0})
    mb.add_stream("S2", ["Água", "Etanol"], 1, flow_rate=100.0, compositions={"Água": 0.0, "Etanol":
1.0})
    mb.add_stream("S3", ["Água", "Etanol"], -1)

    results = mb.get_results()
    s3 = results["S3"]
    print(f"S1 (In): 100 kg/h [Água: 1.0]")
    print(f"S2 (In): 100 kg/h [Etanol: 1.0]")
    print(f"S3 (Out): Flow={s3['flow_rate']:.2f} kg/h, Comp={s3['compositions']}")

def demo_reactor():
    print_header("2. Validação Reator de Conversão")
    # Caso: Reação A -> B com 40% de conversão
    # Alimentação: 100 mol/s de A puro
    mb = MassBalance(components=["A", "B"])
    mb.add_stream("Feed", ["A", "B"], 1, flow_rate=100.0, compositions={"A": 1.0, "B": 0.0})
    mb.add_stream("Product", ["A", "B"], -1)

    # Reação: A -> B (Estequiometria: -1 A, +1 B)
    # Conversão de A = 0.40
    mb.add_reaction({"A": -1, "B": 1}, key_component="A", conversion=0.40)

    results = mb.get_results()
    prod = results["Product"]
    print(f"Feed (In): 100 mol/s [A: 1.0]")
    print(f"Reaction: A -> B (X=40%)")
    print(f"Product (Out): Flow={prod['flow_rate']:.2f} mol/s, Comp={prod['compositions']}")

def demo_recycle():
    print_header("3. Validação Reciclo com Purga")

    mb = MassBalance(components=["A", "B"])
    mb.add_stream("Feed", ["A", "B"], 1, flow_rate=100.0, compositions={"A": 1.0})

    mb_split = MassBalance(components=["A", "B"])
    mb_split.add_stream("S_Main", ["A", "B"], 1, flow_rate=100.0, compositions={"A": 0.5, "B": 0.5})
    mb_split.add_stream("S_Recycle", ["A", "B"], -1)
    mb_split.add_stream("S_Product", ["A", "B"], -1)

    # Split: 20% Reciclo, 80% Produto
    mb_split.add_split("S_Main", "S_Recycle", "S_Product", fraction=0.2)

    res = mb_split.get_results()
    rec = res["S_Recycle"]
    prod = res["S_Product"]

    print(f"Main (In): 100 kg/h [A:0.5, B:0.5]")
    print(f"Split Fraction: 0.2 (20% Reciclo)")
    print(f"Recycle (Out): Flow={rec['flow_rate']:.2f}, Comp={rec['compositions']}")
    print(f"Product (Out): Flow={prod['flow_rate']:.2f}, Comp={prod['compositions']}")

if __name__ == "__main__":
    demo_mixer()
    print("-" * 30)
    demo_reactor()
    print("-" * 30)
    demo_recycle()

```

Fonte: Autoria própria.

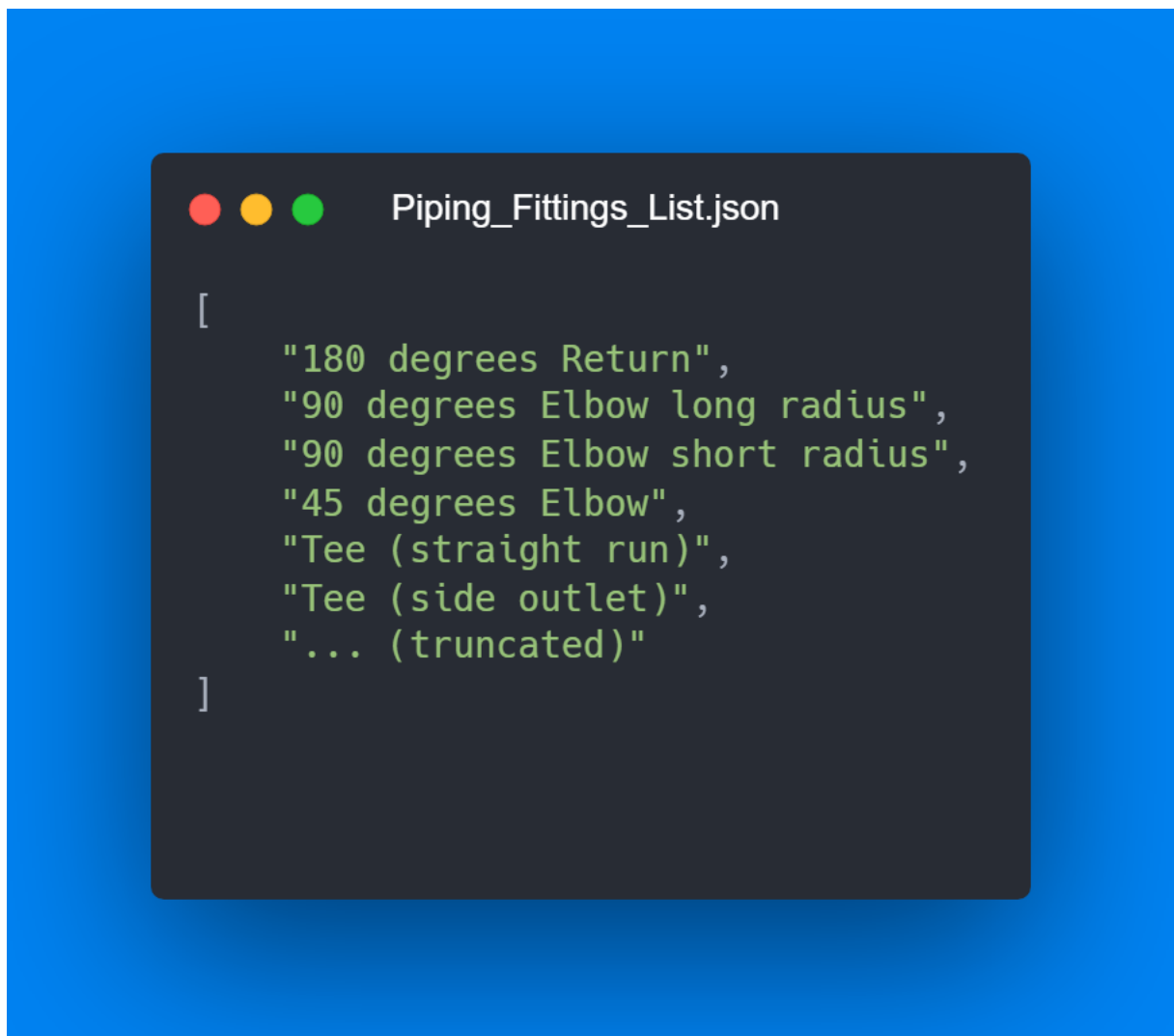
4.4.4 Tubulações

O módulo de tubulações (*Piping*) atua como um banco de dados de engenharia, fornecendo especificações padronizadas para acessórios, materiais e dimensões de tubos. A validação deste módulo consiste na verificação da conformidade dos dados retornados com as tabelas normativas padrão.

4.4.4.1 Acessórios

A função *fittings()* retorna uma lista de todos os acessórios hidráulicos cadastrados no sistema. Esta funcionalidade permite que outros módulos (como o de Perda de Carga) validem a entrada de dados do usuário e utilizem os coeficientes corretos.

Figura 51 – Lista parcial de acessórios disponíveis (JSON)

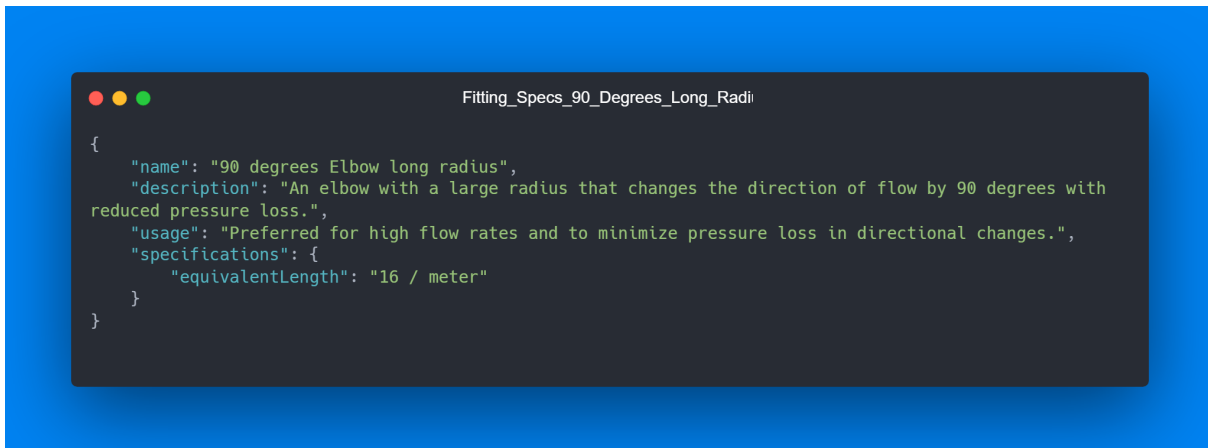


Fonte: Autoria própria.

4.4.4.2 Especificação de Acessórios

A função *fitting_specifications(nome)* retorna os detalhes técnicos de um acessório, incluindo seu comprimento equivalente adimensional (L_{eq}/D) ou coeficiente de perda de carga (K). Para validação, utilizou-se o acessório "90 degrees Elbow long radius"(Cotovelo 90º Raio Longo).

Figura 52 – Especificações do acessório Cotovelo 90º Raio Longo (JSON)



Fonte: Autoria própria.

4.4.4.3 Composição

A função *compositions()* lista os materiais de tubulação disponíveis, associando-os às suas respectivas rugosidades absolutas, parâmetro crítico para o cálculo do Fator de Atrito de Darcy.

Figura 53 – Lista parcial de materiais de tubulação (JSON)



Fonte: Autoria própria.

4.4.4.4 Especificação da Composição

A função *composition_specifications(nome)* retorna as propriedades de um material específico. Para validação, verificou-se o material "Commercial steel"(Aço comercial).

Figura 54 – Especificações do material Aço Comercial (JSON)



Fonte: Autoria própria.

4.4.4.5 Schedules

A função *schedules()* fornece as classes de pressão/espessura disponíveis (ex: SCH40, SCH80), permitindo a seleção adequada conforme a norma ANSI/ASME B36.10M.

Figura 55 – Lista de classes de pressão (Schedules) disponíveis (JSON)



Fonte: Autoria própria.

4.4.4.6 Diâmetros

A função *diameters(schedule)* lista os diâmetros nominais (DN) disponíveis para uma determinada classe (*schedule*).

Figura 56 – Lista parcial de diâmetros para SCH40 (JSON)



```
{
  "6": {
    "nominal_diameter": 6,
    "external_diameter": 10.3,
    "units": "mm"
  },
  "8": {
    "nominal_diameter": 8,
    "external_diameter": 13.7,
    "units": "mm"
  },
  "10": {
    "nominal_diameter": 10,
    "external_diameter": 17.145,
    "units": "mm"
  },
  "15": {
    "nominal_diameter": 15,
    "external_diameter": 21.3,
    "units": "mm"
  },
  "20": {
    "nominal_diameter": 20,
    "external_diameter": 26.9,
    "units": "mm"
  },
  "25": {
    "nominal_diameter": 25,
    "external_diameter": 33.7,
    "units": "mm"
  },
  "...": "(truncated)"
}
```

Fonte: Autoria própria.

4.4.4.7 Especificação de Diâmetros

A função *diameter_specifications(schedule, dn)* retorna as dimensões reais do tubo, incluindo diâmetro externo, espessura de parede e diâmetro interno (calculado). Para validação, utilizou-se um tubo de *Schedule 40* com Diâmetro Nominal 50 (aprox. 2 polegadas).

Figura 57 – Especificações do tubo SCH40 DN50 (JSON)



Fonte: Autoria própria.

4.4.5 Reatores

Esta seção apresenta a validação detalhada dos modelos de reatores CSTR (*Continuous Stirred-Tank Reactor*) e PFR (*Plug Flow Reactor*). Para garantir a precisão dos algoritmos, foram realizados cálculos manuais passo a passo para cenários de projeto e desempenho, comparando-os com os resultados obtidos pela ferramenta computacional.

Para todos os casos, adotou-se uma reação irreversível de primeira ordem $A \rightarrow B$ ocorrendo em fase líquida isotérmica, com os seguintes parâmetros globais:

- Constante cinética da reação (k): $0,05 \text{ s}^{-1}$
- Concentração inicial do reagente A (C_{A0}): 1000 mol/m^3
- Vazão volumétrica de alimentação (Q ou v_0): $0,01 \text{ m}^3/\text{s}$
- Taxa molar de alimentação (F_{A0}): 10 mol/s

4.4.5.1 CSTR: Reator Tanque Agitado Continuamente

A equação de projeto (balanço molar) para um CSTR operando em regime estacionário é a Equação 4.10.

Para uma reação de primeira ordem em fase líquida (densidade constante), a taxa de reação é dada por:

$$-r_A = kC_A = kC_{A0}(1 - X) \quad (4.45)$$

Substituindo na equação de projeto e sabendo que $F_{A0} = C_{A0}v_0$:

$$V_{CSTR} = \frac{C_{A0}v_0X}{kC_{A0}(1 - X)} = \frac{v_0X}{k(1 - X)} \quad (4.46)$$

Cálculo Manual (Design): Considerando uma conversão alvo de $X = 0,85$ (85%):

$$V_{manual} = \frac{0,01 \cdot 0,85}{0,05 \cdot (1 - 0,85)} \quad (4.47)$$

$$V_{manual} = \frac{0,0085}{0,05 \cdot 0,15} = \frac{0,0085}{0,0075} \approx 1,1333 \text{ m}^3 \quad (4.48)$$

Comparação com o Código: O módulo *Reactor* foi executado com os mesmos parâmetros.

Tabela 4 – Validação CSTR ($X = 0,85$)

Parâmetro	Cálculo Manual	<i>Software</i>	Erro (%)
Volume (m^3)	1,1333	1,1333	0,0000

Fonte: Autoria própria.

Também validou-se o cálculo inverso (Desempenho), onde fornece-se $V = 1,1333m^3$ e espera-se $X = 0,85$. O código retornou exatamente 0,8500.

4.4.5.2 PFR: Reator Tubular

A equação de projeto para um PFR é obtida através do balanço diferencial de massa (Equação 4.14).

Substituindo a cinética de primeira ordem:

$$V = F_{A0} \int_0^X \frac{dX}{kC_{A0}(1 - X)} = \frac{F_{A0}}{kC_{A0}} \int_0^X \frac{dX}{1 - X} \quad (4.49)$$

Como $\frac{F_{A0}}{C_{A0}} = v_0$:

$$V = \frac{v_0}{k} [-\ln(1 - X)]_0^X = -\frac{v_0}{k} \ln(1 - X) \quad (4.50)$$

Cálculo Manual (Design): Considerando uma conversão alvo de $X = 0,90$ (90%):

$$V_{manual} = -\frac{0,01}{0,05} \ln(1 - 0,90) \quad (4.51)$$

$$V_{manual} = -0,2 \cdot \ln(0,10) \quad (4.52)$$

Sabendo que $\ln(0,10) \approx -2,302585$:

$$V_{manual} = -0,2 \cdot (-2,302585) \approx 0,4605 \text{ m}^3 \quad (4.53)$$

Comparação com o Código:

Tabela 5 – Validação PFR ($X = 0,90$)

Parâmetro	Cálculo Manual	Software	Erro (%)
Volume (m^3)	0,4605	0,4605	0,0000

Fonte: Autoria própria.

4.4.5.3 PFR com Reciclo

Em reatores com reciclo, parte da corrente de saída retorna à entrada. Para uma razão de reciclo R (vazão de reciclo / vazão de saída do sistema), a conversão na entrada do reator (X_{in}) é diferente de zero, pois há mistura de reagentes frescos ($X = 0$) com reagentes reciclados ($X = X_{out}$).

A relação de balanço de massa no ponto de mistura fornece:

$$X_{in} = \frac{R}{R+1} X_{out} \quad (4.54)$$

A equação de volume torna-se:

$$V = (R+1)F_{A0} \int_{X_{in}}^{X_{out}} \frac{dX}{-r_A} \quad (4.55)$$

Integrando para primeira ordem:

$$V = (R+1) \frac{v_0}{k} \ln \left(\frac{1 - X_{in}}{1 - X_{out}} \right) \quad (4.56)$$

Cálculo Manual (Design): Considerando uma conversão de saída $X_{out} = 0,90$ e razão de reciclo $R = 2,0$.

Passo 1: Calcular X_{in} .

$$X_{in} = \frac{2,0}{2,0+1} \cdot 0,90 = \frac{2}{3} \cdot 0,90 = 0,60 \quad (4.57)$$

Passo 2: Calcular o Volume.

$$V_{manual} = (2+1) \cdot \frac{0,01}{0,05} \cdot \ln \left(\frac{1 - 0,60}{1 - 0,90} \right) \quad (4.58)$$

$$V_{manual} = 3 \cdot 0,2 \cdot \ln \left(\frac{0,40}{0,10} \right) = 0,6 \cdot \ln(4) \quad (4.59)$$

Sabendo que $\ln(4) \approx 1,38629$:

$$V_{\text{manual}} = 0,6 \cdot 1,38629 \approx 0,8318 \text{ m}^3 \quad (4.60)$$

Comparação com o Código:

Tabela 6 – Validação PFR com Reciclo ($R = 2,0$; $X = 0,90$)

Parâmetro	Cálculo Manual	<i>Software</i>	Erro (%)
Volume (m^3)	0,8318	0,8318	0,0000

Fonte: Autoria própria.

Os testes confirmam que as rotinas computacionais implementam corretamente as equações diferenciais e algébricas fundamentais da engenharia de reações, incluindo cenários complexos de integração numérica com limites variáveis como no caso do reciclo.

5 Resultados e Discussões

O desenvolvimento do *software* demonstrou que é possível integrar conceitos fundamentais da engenharia química com tecnologias modernas de programação para produzir uma ferramenta acessível, modular e capaz de automatizar cálculos recorrentes da área.

Do ponto de vista funcional, o *backend* em *FastAPI* apresentou desempenho satisfatório, respondendo rapidamente aos cálculos mesmo em operações envolvendo chamadas reiteradas ao módulo de propriedades termodinâmicas (*CoolProp*) ou a rotinas numéricas como métodos iterativos de Brent e integrações do *SciPy*.

A validação de entradas pelo *Pydantic* mostrou-se fundamental para evitar inconsistências numéricas e garantir segurança nos cálculos, reduzindo significativamente erros comuns como unidades incorretas, tipos inválidos ou parâmetros ausentes.

No *frontend*, a utilização de HTML, CSS, *JavaScript* modular e bibliotecas como *Tailwind* e *Select2* resultou em uma interface simples, leve e responsiva, adequada ao propósito educacional e profissional. A modularidade do *JavaScript* permitiu encapsular cada categoria de cálculo em classes específicas, facilitando manutenção e futura expansão para novos módulos, sendo um ponto essencial em um projeto de código aberto.

A comparação dos resultados obtidos com valores de referência da literatura e com *softwares* consolidados evidenciou boa concordância para todos os módulos implementados. Cálculos de *Reynolds*, perdas de carga, seleção de diâmetros e estimativas de *NPSH* apresentaram diferenças mínimas em relação às tabelas clássicas. Para reatores CSTR e PFR, os valores de conversão e volume mostraram-se consistentes com soluções de Fogler e Levenspiel, reforçando a confiabilidade das integrações numéricas.

Embora o *software* esteja plenamente funcional para os módulos desenvolvidos, observou-se que o sistema ainda pode ser expandido significativamente. A ausência de módulos como trocadores de calor, destiladores, absorção, secagem e operações com múltiplas reações limitam sua aplicabilidade em projetos de maior complexidade. Do ponto de vista de usabilidade, futuras melhorias incluem exportação de relatórios, gráficos mais sofisticados e integração com bancos de dados externos de propriedades físicas.

Ainda assim, o objetivo de criar uma ferramenta precisa, acessível, gratuita e colaborativa foi alcançado. O fato de o projeto ser *open-source* torna natural que seu ciclo de evolução dependa da contribuição da comunidade acadêmica e profissional, que poderá incluir novos módulos, otimizar algoritmos e expandir a base de dados.

6 Conclusões

O presente trabalho alcançou com êxito o objetivo geral de desenvolver um *software web open-source* para dimensionamento de operações unitárias em engenharia química, integrando rotinas de cálculo baseadas em equações clássicas com uma arquitetura cliente-servidor moderna e acessível via API. Os módulos implementados, incluindo dimensionamento de tubulações, reatores CSTR e PFR, balanços de massa, análises de escoamento e propriedades termofísicas, demonstraram precisão e utilidade prática, validados utilizando referências consolidadas na literatura.

A presente ferramenta possibilita o futuro preenchimento de uma lacuna no acesso à simulações didáticas e gratuitas, especialmente em contextos acadêmicos e para profissionais autônomos, promovendo o aprendizado conceitual e a automação de cálculos rotineiros. O caráter *open-source* é fundamental para sua completude futura, permitindo contribuições comunitárias que expandam o repertório de módulos e aprimorem sua robustez, tornando-o uma plataforma escalável para o ensino e a prática da engenharia química na era da Indústria 4.0.

Como perspectivas, sugere-se a adição de funcionalidades como simulações multifásicas, otimização de processos e integração com bancos de dados remotos e associando ao Padrão de Interface CAPE-OPEN, consolidando o *software* como uma alternativa viável a ferramentas proprietárias.

Referências

ALVES, G. N. et al. Uso do software aberto dwsim na simulação de biorrefinarias: desafios e aprendizados. In: *Anais do XXX Congresso de Iniciação Científica da UNICAMP*. Campinas: UNICAMP, 2022.

Aspen Technology. *Aspen Plus*. Bedford, MA, 2025. Disponível em: <<https://www.aspentech.com/en/products/engineering/aspen-plus>>.

BELL, I. H. et al. Pure and pseudo-pure fluid thermophysical property evaluation and the open-source thermophysical property library coolprop. *Industrial & Engineering Chemistry Research*, ACS Publications, v. 53, n. 6, p. 2498–2508, 2014.

Chemstations. *CHEMCAD Process Simulation Software*. Houston, TX, 2025. Disponível em: <<https://www.chemstations.com/CHEMCAD>>.

COLEBROOK, C. F. Turbulent flow in pipes, with particular reference to the transition region between the smooth and rough pipe laws. *Journal of the Institution of Civil Engineers*, Thomas Telford-ICE Virtual Library, v. 11, n. 4, p. 133–156, 1939.

DWSIM. *DWSIM – Open-Source Chemical Process Simulator (v.8.0)*. 2025. Acesso em: 10 nov. 2025. Disponível em: <<http://dwsim.org>>.

FELDER, R. M.; ROUSSEAU, R. W. *Princípios Elementares dos Processos Químicos*. 3. ed. Rio de Janeiro: LTC, 2005.

FOGLER, H. S. *Elementos de Engenharia das Reações Químicas*. 4. ed. Rio de Janeiro: LTC, 2009.

HAALAND, S. E. Simple and explicit formulas for the friction factor in turbulent pipe flow. *Journal of Fluids Engineering*, American Society of Mechanical Engineers, v. 105, n. 1, p. 89–90, 1983.

LEVENSPIEL, O. *Engenharia das Reações Químicas*. 3. ed. São Paulo: Edgard Blücher, 2000.

PERRY, R. H.; GREEN, D. W. (Ed.). *Perry's Chemical Engineers' Handbook*. 8. ed. New York: McGraw-Hill, 2007.

SWAMEE, P. K.; JAIN, A. K. Explicit equations for pipe-flow problems. *Journal of the Hydraulics Division*, ASCE, v. 102, n. 5, p. 657–664, 1976.

WHITE, F. M. *Mecânica dos Fluidos*. 8. ed. Porto Alegre: AMGH, 2018.